Applied Cryptography: How It Works

Contents

Notation (list of symbols) 6

1 Block ciphers vs. stream ciphers 8
- 1.1 Symmetric encryption in one line . . . 8
- 1.2 Block ciphers . . . 8
- 1.3 Stream ciphers . . . 8
- 1.4 Side by side . . . 9
- 1.5 Symbols . . . 9

2 Modes of operation 11
- 2.1 ECB — and why never to use it . . . 11
- 2.2 CBC — cipher block chaining . . . 11
- 2.3 CTR — counter mode . . . 12
- 2.4 Padding the last block . . . 12
- 2.5 These modes hide, but do not protect . . . 12
- 2.6 Symbols . . . 13

3 Authenticated encryption 14
- 3.1 Two goals . . . 14
- 3.2 The tool: a MAC . . . 14
- 3.3 Order matters . . . 14
- 3.4 The AEAD interface . . . 14
- 3.5 AES-GCM . . . 15
- 3.6 ChaCha20-Poly1305 . . . 15
- 3.7 Symbols . . . 16

4 Inside GHASH 17
- 4.1 A finite field: GF(2^{128}) . . . 17
- 4.2 GHASH = a polynomial at a secret point . . . 17
- 4.3 A universal hash is not yet a MAC . . . 17
- 4.4 Wegman–Carter: mask with a one-time pad . . . 18
- 4.5 Why nonce reuse recovers H . . . 18
- 4.6 Poly1305 is the same recipe . . . 18
- 4.7 Symbols . . . 18

5 Inside ChaCha20 19
- 5.1 The state: a 4×4 grid of words . . . 19
- 5.2 The quarter-round . . . 20
- 5.3 Rounds: columns, then diagonals . . . 20
- 5.4 Feed-forward, then the keystream . . . 21
- 5.5 Why ARX . . . 21
- 5.6 Symbols . . . 22

6 Inside AES 23
- 6.1 The state and the rounds . . . 23
- 6.2 SubBytes — the substitution (confusion) . . . 23
- 6.3 ShiftRows — shuffle across columns (diffusion) . . . 24

6.4 MixColumns — mix within a column (diffusion) . . . 24
6.5 AddRoundKey and the key schedule . . . 24
6.6 SP-network vs. ARX . . . 25
6.7 Symbols . . . 25

7 Nonce management **26**
7.1 Why a repeated nonce is a disaster . . . 26
7.2 Why $P \oplus P'$ is already a break . . . 26
7.3 Two ways to keep nonces unique . . . 27
7.4 The birthday bound on random nonces . . . 27
7.5 Bigger nonces: XChaCha20 . . . 27
7.6 Surviving reuse: misuse-resistant AEAD . . . 27
7.7 Which to use . . . 28
7.8 Symbols . . . 28

8 Diffie–Hellman key exchange **29**
8.1 The idea: mixing that will not un-mix . . . 29
8.2 The exchange . . . 29
8.3 A worked example . . . 30
8.4 From shared secret to key . . . 30
8.5 The catch: it does not authenticate . . . 30
8.6 In practice: ECDH and forward secrecy . . . 30
8.7 Symbols . . . 31

9 Digital signatures **32**
9.1 Key pairs: sign with private, verify with public . . . 32
9.2 Sign the hash, not the message . . . 32
9.3 How it works: RSA and elliptic curves . . . 32
9.4 Certificates: trusting a public key . . . 33
9.5 Authenticating Diffie–Hellman . . . 33
9.6 Symbols . . . 33

10 Putting it together: a secure channel **35**
10.1 The handshake . . . 35
10.2 Deriving the session keys . . . 36
10.3 The data phase . . . 36
10.4 Where each guarantee comes from . . . 36
10.5 Symbols . . . 37

11 A post-quantum note **38**
11.1 What a quantum computer breaks . . . 38
11.2 Harvest now, decrypt later . . . 38
11.3 The replacements . . . 38
11.4 Hybrid, for now . . . 39
11.5 Symbols . . . 39

12 Randomness **40**
12.1 Random means unpredictable . . . 40
12.2 Entropy plus a CSPRNG . . . 40

12.3 When randomness fails . . . 40
12.4 In practice . . . 41
12.5 Symbols . . . 41

13 Public-key encryption **42**
13.1 The third public-key tool . . . 42
13.2 Textbook RSA, and why it needs padding . . . 42
13.3 Why you don't encrypt the data directly . . . 42
13.4 Hybrid encryption . . . 42
13.5 The modern shape: KEM + DEM . . . 43
13.6 Who sent it? . . . 43
13.7 Symbols . . . 43

14 Inside elliptic curves (ECDH) **44**
14.1 Why elliptic curves . . . 44
14.2 Adding points: the group . . . 44
14.3 Scalar multiplication: the one-way step . . . 45
14.4 ECDH . . . 46
14.5 Curve25519 / X25519 in practice . . . 46
14.6 Symbols . . . 47

15 Side-channel attacks **48**
15.1 Attacking the implementation, not the algorithm . . . 48
15.2 Timing attacks . . . 48
15.3 Cache-timing attacks . . . 48
15.4 Power and electromagnetic analysis . . . 49
15.5 The defence: constant-time . . . 49
15.6 Symbols . . . 49

16 Key management **50**
16.1 The forgotten layer . . . 50
16.2 Generating keys . . . 50
16.3 Storage: the key hierarchy . . . 50
16.4 Rotation and destruction . . . 51
16.5 Principles . . . 51
16.6 Symbols . . . 51

17 The padding-oracle attack **52**
17.1 The setup: CBC, padding, and an oracle . . . 52
17.2 The key idea: control the previous block . . . 52
17.3 Cracking the last byte . . . 52
17.4 Cracking the rest . . . 52
17.5 Why it works, and the fix . . . 53
17.6 Symbols . . . 53

18 The 25519 curves: X25519 and Ed25519 **54**
18.1 Why a named curve . . . 54
18.2 One curve, two forms . . . 54
18.3 X25519: key agreement . . . 54

18.4 ECDSA: the scheme Ed25519 improves on . . . 55
18.5 EdDSA and Ed25519: deterministic signatures . . . 55
18.6 Symbols . . . 56

19 TLS and the web PKI **57**
19.1 TLS: the protocol around the handshake . . . 57
19.2 The web PKI: who to trust . . . 57
19.3 The weak link, and the fixes . . . 58
19.4 Symbols . . . 58

20 NIST and the standards landscape **59**
20.1 What NIST publishes . . . 59
20.2 The open-competition model . . . 59
20.3 The trust question . . . 59
20.4 NIST, or not? . . . 59
20.5 Symbols . . . 60

21 Modular arithmetic **61**
21.1 Wrapping at n . . . 61
21.2 The operations . . . 61
21.3 Inverses and fields . . . 61
21.4 Why crypto uses it . . . 61
21.5 Symbols . . . 61

22 The discrete logarithm problem **62**
22.1 Symbols . . . 62

23 The integer factorization problem **63**
23.1 Symbols . . . 63

24 Shor's algorithm **64**
24.1 Step 1: factoring becomes order-finding . . . 64
24.2 Step 2: the quantum period-finder . . . 64
24.3 Step 3: read off the period . . . 66
24.4 Discrete log falls the same way . . . 66
24.5 Why it is fast — and why your keys are still safe today . . . 66
24.6 Symbols . . . 67

25 Grover's algorithm **68**
25.1 The problem: unstructured search . . . 68
25.2 How it works: amplitude amplification . . . 68
25.3 The geometry: it is just a rotation . . . 69
25.4 What it means for crypto . . . 69
25.5 Symbols . . . 70

26 Quantum key distribution **71**
26.1 The idea: secrecy from physics . . . 71
26.2 BB84, step by step . . . 71
26.3 Why Eve gets caught . . . 72

26.4 From sifted bits to a usable key 72
26.5 QKD versus post-quantum cryptography 73
26.6 Symbols 73

27 Lattice-based cryptography **74**
27.1 What is a lattice? 74
27.2 The hard problems: SVP and CVP 75
27.3 Learning With Errors (LWE) 75
27.4 Encryption from LWE (Regev) 76
27.5 Making it practical: Ring and Module LWE 77
27.6 Symbols 77

28 ML-KEM (Kyber) **78**
28.1 The interface 78
28.2 The ring it lives in 78
28.3 The encryption inside (K-PKE) 78
28.4 From PKE to KEM: the Fujisaki–Okamoto transform 79
28.5 Parameters and practicalities 79
28.6 Symbols 80

29 ML-DSA (Dilithium) **81**
29.1 The interface 81
29.2 Keys: a Module-LWE instance 81
29.3 Sign and verify: Fiat–Shamir with aborts 81
29.4 Why the aborts 82
29.5 Parameters and cost 82
29.6 Symbols 82

30 SLH-DSA (SPHINCS$^+$) **83**
30.1 The brick: a one-time signature 83
30.2 Many signatures from one key: the Merkle tree 83
30.3 Going stateless: random index, hypertree, FORS 84
30.4 The whole signature 84
30.5 Cost, and when to reach for it 84
30.6 Symbols 85

31 Zero-knowledge proofs **86**
31.1 The three properties 86
31.2 The intuition: the cave 86
31.3 A real one: Schnorr's protocol 87
31.4 Non-interactive proofs: Fiat–Shamir 87
31.5 Modern zero-knowledge: SNARKs and STARKs 87
31.6 Symbols 88

Notation (list of symbols)

k	secret key, shared by encryptor and decryptor (symmetric).
P	plaintext: the message to encrypt, a bit string.
C	ciphertext: the encrypted output.
b	block size in bits (e.g. 128 for AES).

1 Block ciphers vs. stream ciphers

A cryptographic hash is *one-way*: you compute a digest and can never recover the message. *Encryption* is the opposite — reversible, with a secret key — and its goal is *confidentiality*: scramble a message so that only a key-holder can read it.

1.1 Symmetric encryption in one line

Symmetric means a single shared secret key k runs both directions:

$$C = \mathrm{Enc}_k(P), \qquad P = \mathrm{Dec}_k(C).$$

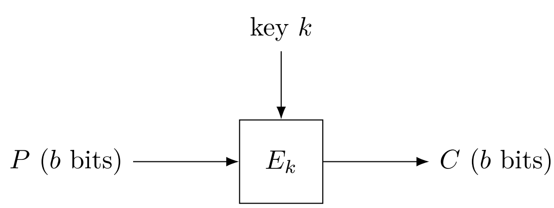
P is the *plaintext* (the message, a bit string), C the *ciphertext* (the scrambled output), and k the key. Dec_k is the exact inverse of Enc_k, so decrypting what you encrypted returns the original. Two families build Enc: *block* ciphers and *stream* ciphers.

1.2 Block ciphers

A block cipher encrypts a *fixed-size block* of bits at a time. Formally it is a keyed *permutation* — a reversible shuffle —

$$E_k : \{0,1\}^b \to \{0,1\}^b,$$

where b is the block size (AES uses $b = 128$ bits). "Permutation" means that for a fixed key, E_k is a bijection on the 2^b possible blocks (one-to-one and onto), so it has an inverse $D_k = E_k^{-1}$ that decryption uses. AES, and the retired DES, are block ciphers.



One block is only b bits, so to encrypt a longer message you need a *mode of operation*: a recipe for chaining many block-cipher calls (CBC, CTR, GCM, . . .). Modes usually also require *padding* to fill out the final partial block. (The naive mode, ECB, encrypts each block independently and leaks repeated-block patterns — never use it.)

1.3 Stream ciphers

A stream cipher instead generates a long pseudorandom *keystream* from the key and a *nonce*, and combines it with the plaintext one bit (or byte) at a time with XOR:

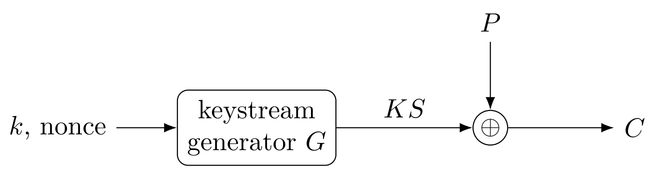
$$KS = G(k, \mathrm{nonce}), \qquad C_i = P_i \oplus KS_i, \qquad P_i = C_i \oplus KS_i.$$

The jargon:

- $\oplus$ (*XOR*, exclusive-or): a bit operation that outputs 1 exactly when its two input bits differ. Its key property is being self-inverse, $x \oplus y \oplus y = x$ — which is why XORing the ciphertext with the *same* keystream gives the plaintext back.
- *keystream* KS: the pseudorandom bit sequence the cipher produces to mask the message.

- *nonce* ("number used once", a.k.a. IV): a non-secret value that makes each encryption under the same key produce a *different* keystream. It must never repeat for a given key (see the pitfall below).

This is a practical *one-time pad*: the one-time pad XORs the message with a *truly random* pad as long as the message — perfectly secret but impractical (the key is as big as the data and usable once); a stream cipher replaces that true-random pad with a pseudorandom keystream stretched from a short key. ChaCha20 is the modern choice; RC4 is the well-known broken one.



1.4 Side by side

	Block cipher	Stream cipher
Unit	fixed b-bit block	bit / byte at a time
Mechanism	keyed permutation E_k	XOR with a keystream
Long messages	need a mode (CBC, CTR, GCM)	naturally continuous
Padding	usually required	none
Examples	AES, DES (retired)	ChaCha20, RC4 (broken)
Random access	CTR yes, CBC no	yes

The line blurs. Run a block cipher in *counter (CTR)* mode — encrypt a running counter to make a keystream $KS_i = E_k(\text{nonce} \parallel i)$ and XOR it into the plaintext — and the block cipher *becomes* a stream cipher. So "block vs. stream" is partly about the underlying primitive and partly about how you use it.

The one fatal pitfall: never reuse a (key, nonce) pair. If two messages are XORed with the *same* keystream, then

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

the keystream cancels and the attacker learns $P \oplus P'$ — usually enough to unravel both messages. (This is the "two-time pad" break.) A fresh nonce per message avoids it.

1.5 Symbols

New persistent symbols this section — k, P, C, b — are in the global list of symbols. The symbols below are *local to this section.*

Local symbol	Meaning (this section)
$\mathrm{Enc}_k, \mathrm{Dec}_k$	encrypt / decrypt under key k (exact inverses)
E_k, D_k	block-cipher permutation and its inverse, on b-bit blocks
G	keystream generator
KS	keystream (pseudorandom bits); KS_i is its i-th unit
nonce	"number used once": non-secret, must not repeat per key
$\oplus$	bitwise XOR
P_i, C_i	i-th bit/byte of plaintext / ciphertext
i	position index

2 Modes of operation

A block cipher E_k encrypts exactly one b-bit block (§1). Real messages are longer, shorter, or not a clean multiple of b. A *mode of operation* is the recipe for calling E_k repeatedly to encrypt a message of any length. Throughout, pad the message (§2.4) and split it into n blocks

$$P_1, P_2, \ldots, P_n, \qquad \text{each } b \text{ bits},$$

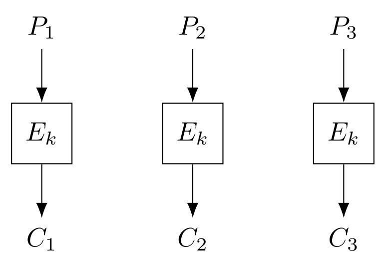
encrypting to ciphertext blocks $C_1, \ldots, C_n$. The modes differ in *how* each block is fed through E_k — and that choice decides everything about safety and speed.

2.1 ECB — and why never to use it

The obvious recipe, *Electronic CodeBook* (ECB), encrypts each block on its own:

$$C_i = E_k(P_i), \qquad P_i = D_k(C_i),$$

where $D_k = E_k^{-1}$ is the inverse permutation (§1).



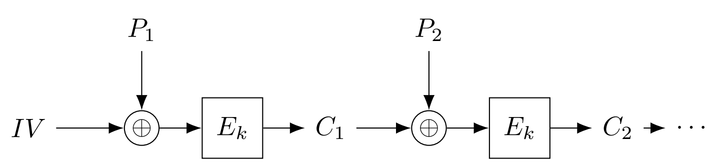
Because E_k is deterministic, *equal plaintext blocks always give equal ciphertext blocks*: if $P_i = P_j$ then $C_i = C_j$. So the ciphertext still shows wherever the message repeats. Encrypt a bitmap image this way and the picture stays recognisable — the infamous "ECB penguin." ECB leaks structure; never use it for real data. The fix in every mode below is to make encryption *randomised* so identical blocks diverge.

2.2 CBC — cipher block chaining

CBC removes the leak by XORing each plaintext block with the *previous ciphertext block* before encrypting, seeding the chain with a random *initialization vector IV*:

$$C_i = E_k\big(P_i \oplus C_{i-1}\big), \qquad C_0 \equiv IV.$$

The IV is a non-secret, random b-bit value chosen fresh per message (like the nonce of §1); for CBC it must be *unpredictable*. Because the IV changes, the same plaintext encrypts to different ciphertext each time — the randomisation ECB lacked.



To decrypt, run D_k and undo the XOR. Starting from $C_i = E_k(P_i \oplus C_{i-1})$, apply D_k to both sides: $D_k(C_i) = P_i \oplus C_{i-1}$; then XOR C_{i-1} and use $x \oplus y \oplus y = x$:

$$P_i = D_k(C_i) \oplus C_{i-1}.$$

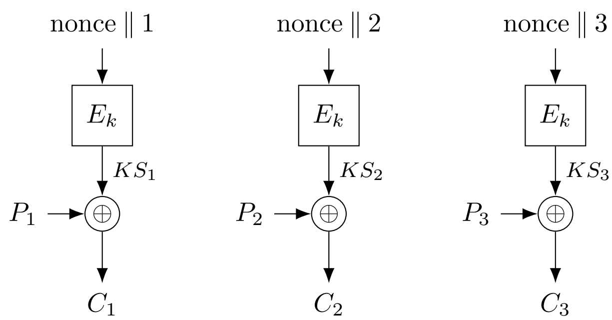
Trade-offs. Encryption is *sequential* — block i needs C_{i-1}, so it cannot be parallelised (decryption can, since all C blocks are already in hand). The message must be padded to a whole number of blocks (below).

2.3 CTR — counter mode

CTR takes the trick from §1: don't encrypt the plaintext at all — encrypt a *counter* to manufacture a keystream, then XOR. For block i,

$$KS_i = E_k(\text{nonce} \,\|\, i), \qquad C_i = P_i \oplus KS_i, \qquad P_i = C_i \oplus KS_i,$$

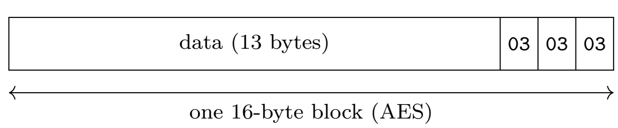
where nonce $\|$ i is the nonce concatenated with the block index i, forming a distinct b-bit input for every block, and KS_i is the resulting keystream block.



Trade-offs. The blocks are independent, so CTR is *fully parallel* and allows *random access* (jump straight to block i). It needs *no padding* (truncate the last keystream block to the message length), and uses only E_k — never the inverse D_k. Encryption and decryption are the *same* operation. The one rule, exactly as in §1: never reuse a $(\text{key}, \text{nonce})$ pair, or the keystream repeats and the "two-time pad" break applies.

2.4 Padding the last block

ECB and CBC need the plaintext to fill whole b-bit blocks; CTR and stream ciphers do not. The standard scheme, *PKCS#7*: if p bytes are missing from the final block, append p bytes each holding the *value* p. The value records how many to strip, so removal is unambiguous. If the message already fills the blocks exactly, append one whole extra block of padding — so there is always something to remove.



Here 13 data bytes leave $p = 3$ to fill a 16-byte block, so three `0x03` bytes are appended.

2.5 These modes hide, but do not protect

ECB, CBC, and CTR give *confidentiality* — they scramble the message — but no *integrity*: nothing detects tampering. CTR makes this stark. Since $P_i = C_i \oplus KS_i$, flipping any bit of C_i flips the *same* bit of the recovered plaintext, with no key needed:

$$(C_i \oplus \Delta) \oplus KS_i = P_i \oplus \Delta.$$

An attacker who knows the message format can thus make targeted edits the decryptor cannot notice. Confidentiality is not integrity. Real systems therefore use *authenticated encryption*, which bolts a message authentication code onto a mode (AES-GCM, ChaCha20-Poly1305) so any change is rejected — the subject of its own section.

2.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
E_k, D_k	block-cipher permutation and its inverse, on b-bit blocks
P_i, C_i	i-th b-bit plaintext / ciphertext block
n	number of blocks after padding
IV	initialization vector: random, non-secret, fresh per message
nonce	"number used once"; with i forms CTR's per-block input
KS_i	i-th keystream block, $E_k(\text{nonce} \parallel i)$ (CTR)
$\oplus$	bitwise XOR
Δ	an attacker's chosen bit-flip mask
i	block index, $1, \ldots, n$

3 Authenticated encryption

The modes of §2 hide a message but do nothing to stop *tampering* — flip a ciphertext bit and the plaintext changes, undetected (§7). *Authenticated encryption with associated data* (AEAD) adds integrity.

3.1 Two goals

- *confidentiality* — an eavesdropper learns nothing (what §2 gave);
- *integrity / authenticity* — tampering is detected, and the message came from a key-holder.

Encryption alone gives only the first.

3.2 The tool: a MAC

A *MAC* (message authentication code) stamps a message with a short *tag*, checked by recompute-and-compare (in *constant time*, so timing leaks nothing):

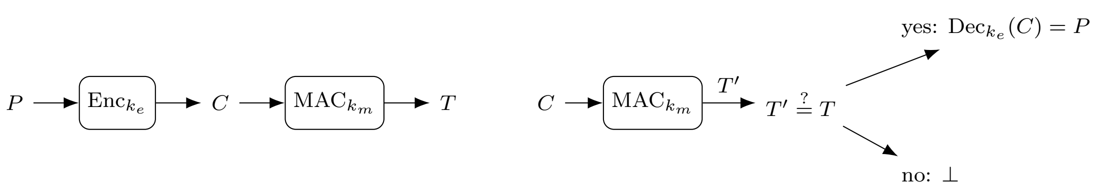
$$T = \mathrm{MAC}_{k_m}(M) \in \{0,1\}^t \ (t = 128), \qquad \text{accept} \iff \mathrm{MAC}_{k_m}(M) = T.$$

Only a holder of k_m can forge a valid tag, and one flipped bit breaks it. (HMAC, GHASH, and Poly1305 are MACs.)

3.3 Order matters

Three ways to combine a cipher and a MAC, with *independent* keys $k_e \neq k_m$:

- **Encrypt-and-MAC** (SSH): $C = \mathrm{Enc}_{k_e}(P)$, $T = \mathrm{MAC}_{k_m}(P)$ — tag is over the plaintext (leaks equality), and you must decrypt to verify.
- **MAC-then-Encrypt** (old TLS): $T = \mathrm{MAC}_{k_m}(P)$, $C = \mathrm{Enc}_{k_e}(P \parallel T)$ — must decrypt attacker data *before* checking T, the opening for padding oracles (§17).
- **Encrypt-then-MAC** (IPsec, recommended): $C = \mathrm{Enc}_{k_e}(P)$, $T = \mathrm{MAC}_{k_m}(C)$ — verify T on C first; a bad tag is $\perp$, no decryption.

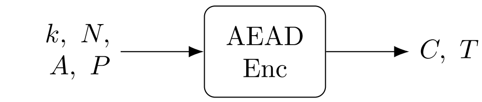


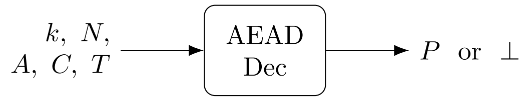
Encrypt-then-MAC (left) and its verify (right): the tag is checked on C before any decryption.

3.4 The AEAD interface

One primitive bakes in a correct composition, so application code cannot pick the wrong order or reuse a key:

$$(C, T) = \mathrm{Enc}_k(N, A, P), \qquad \mathrm{Dec}_k(N, A, C, T) \in \{\, P,\ \perp \}.$$

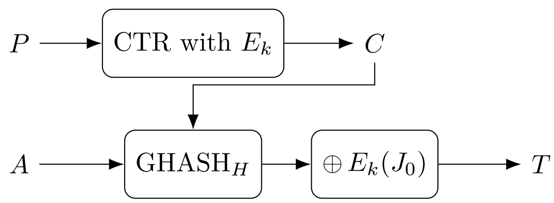




- N — nonce, unique per key (§1); A — associated data: authenticated but *not* encrypted (e.g. a header);
- T — tag, covering *both* C and A; $\perp$ — "reject." Dec verifies first, so a forgery never gets decrypted.

3.5 AES-GCM

CTR (confidentiality, §2) + GHASH (a universal hash over $\mathrm{GF}(2^{128})$, keyed by $H = E_k(0^{128})$; detailed in §4).



GHASH folds the blocks X_i of $A \parallel C \parallel \text{len}$ Horner-style, then is masked into the tag (J_0 = initial counter from N):

$$Y_0 = 0, \qquad Y_i = (Y_{i-1} \oplus X_i) \cdot H, \qquad T = Y_{\text{last}} \oplus E_k(J_0).$$

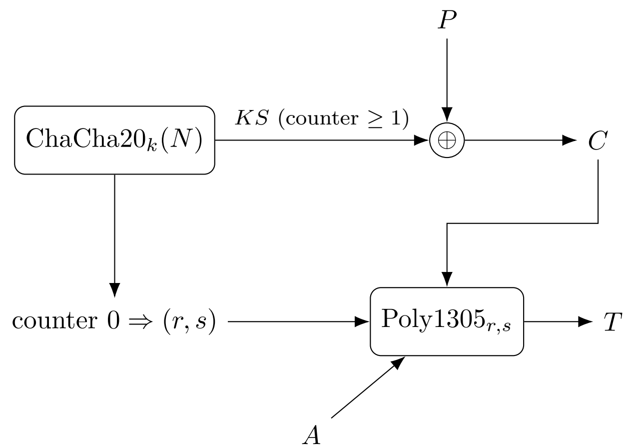
Send $N \parallel A \parallel C \parallel T$. The receiver recomputes T' from A, C only (never P); if $T' \neq T \to \perp$, else CTR-decrypt — verify-before-decrypt.

- The mask $\oplus E_k(J_0)$ is essential: GHASH alone is forgeable; a fresh-per-nonce mask makes the tag a MAC.
- **Nonce reuse is doubly fatal** — it breaks CTR confidentiality *and* leaks H, enabling forgery.

Fast with AES-NI / PCLMULQDQ hardware; NIST SP 800-38D.

3.6 ChaCha20-Poly1305

ChaCha20 (§1) + Poly1305. ChaCha20's keystream comes in counted 64-byte blocks: block 0 gives the *one-time* key (r, s) (its first 32 bytes — r = bytes 0–15, s = 16–31), blocks ≥ 1 give the keystream.



Poly1305 folds the 16-byte blocks c_i of $A \parallel C \parallel$ len Horner-style, mod $p = 2^{130} - 5$:

$$a_0 = 0, \qquad a_i = (a_{i-1} + c_i) \cdot r \bmod p, \qquad T = (a_{\text{last}} + s) \bmod 2^{128}.$$

The key (r, s) is used for *one* message (it is nonce-derived). Transmission and verify-before-decrypt are as in GCM: send $N \parallel A \parallel C \parallel T$, recompute the tag over A, C, then decrypt only on a match. No special hardware, constant-time — the pick without AES acceleration; RFC 8439.

	AES-GCM	ChaCha20-Poly1305
Cipher	AES (block, CTR)	ChaCha20 (stream)
MAC	GHASH, GF(2^{128})	Poly1305, mod $2^{130}-5$
Fastest with	AES-NI hardware	plain software
Nonce reuse	catastrophic	catastrophic
Standard	NIST SP 800-38D	RFC 8439

3.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\text{Enc}_{k_e}, \text{Dec}_{k_e}$	confidentiality-only encrypt / decrypt
MAC_{k_m}	message authentication code; tag $T = \text{MAC}_{k_m}(M)$
$k_e,\ k_m$	independent encryption and MAC keys ($k_e \neq k_m$)
N	nonce: unique per key
A	associated data: authenticated, not encrypted
$T,\ T'$	tag; and the tag recomputed at verification
$\perp$	"reject": returned when the tag fails
H	GCM's GHASH key, $H = E_k(0^{128})$
J_0	GCM's initial counter block, derived from N
$X_i,\ Y_i$	GHASH input blocks and running value
(r, s)	Poly1305 one-time key, from ChaCha20 block 0
$c_i,\ a_i$	Poly1305 input blocks and running value

4 Inside GHASH

AES-GCM (§3) authenticates with GHASH. HMAC got its own treatment (in the hashing collection); GHASH deserves one too. It is a *universal hash* turned into a MAC by a one-time mask — the *Wegman–Carter* recipe, which Poly1305 also follows.

4.1 A finite field: GF(2^{128})

A *field* is a set you can add, subtract, multiply, and divide in (by anything nonzero) — like the rationals or reals, but here *finite*. We have used one already: the integers mod a prime p, written GF(p), where every nonzero value has an inverse (§8, §18).

GF(2^n) is a field with 2^n elements — but *not* the integers mod 2^n (that is not a field: 2 has no inverse mod 4). Instead each element is an n-bit string, read as a polynomial with $\{0, 1\}$ coefficients:

- **add** = XOR (add coefficients mod 2, no carries);
- **multiply** = multiply the polynomials, then reduce modulo a fixed *irreducible* degree-n polynomial — the field's "modulus," playing the role the prime p does in GF(p).

Because that modulus is irreducible, every nonzero element has an inverse, so division works.

	GF(p)	GF(2^n)
Elements	$0, \ldots, p-1$	n-bit strings (polynomials)
Add	mod-p addition	XOR
Multiply	mod-p multiply	carry-less multiply, then reduce
Used in	DH, ECDSA (mod p/n)	AES (2^8), GHASH (2^{128})

GHASH works in GF(2^{128}) (16-byte blocks); AES (§6) uses GF(2^8) (bytes). The 2 means bits; the exponent, how many. CPUs have a carry-less multiply instruction (PCLMULQDQ) for it.

4.2 GHASH = a polynomial at a secret point

The key is $H = E_k(0^{128})$ (the block cipher on the all-zero block). The data blocks $X_1, \ldots, X_m$ (of $A \,\|\, C \,\|\, \text{len}$) are the coefficients of a polynomial, evaluated at H:

$$\text{GHASH}_H(X_1, \ldots, X_m) = X_1 H^m \oplus X_2 H^{m-1} \oplus \cdots \oplus X_m H \qquad (\text{in GF}(2^{128})).$$

In practice it is computed Horner-style, one block at a time:

$$0 \longrightarrow \boxed{\oplus X_1,\ \times H} \longrightarrow \boxed{\oplus X_2,\ \times H} \longrightarrow \cdots \longrightarrow \boxed{\oplus X_m,\ \times H} \longrightarrow Y$$

$$Y_0 = 0, \qquad Y_i = (Y_{i-1} \oplus X_i) \cdot H, \qquad Y = Y_m = \text{GHASH}_H(X).$$

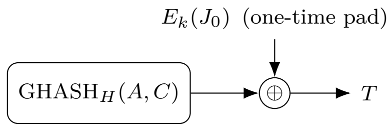
4.3 A universal hash is not yet a MAC

GHASH guarantees only that two *different* messages rarely collide — it is not, by itself, unforgeable. Output $\text{GHASH}_H(M)$ as the tag directly and an attacker can forge: the output is a fixed polynomial in H, so they can shift the message and predict the change. So a universal hash needs a *secret, one-time pad* to become a MAC.

4.4 Wegman–Carter: mask with a one-time pad

GCM masks the GHASH output with a fresh per-nonce pad $E_k(J_0)$ (J_0 = the nonce-derived counter):

$$T = \mathrm{GHASH}_H(A, C) \;\oplus\; E_k(J_0).$$



The pad hides the universal-hash output, so forgery would require guessing it — infeasible. Universal hash $\oplus$ one-time pad is the *Wegman–Carter* MAC.

4.5 Why nonce reuse recovers H

The pad must be fresh. Reuse a nonce and it repeats, so two tags over messages with blocks X, X' cancel it:

$$T \oplus T' = \mathrm{GHASH}_H(X) \oplus \mathrm{GHASH}_H(X'),$$

a polynomial equation of degree $\sim m$ in the single unknown H. Solve it (factor over $\mathrm{GF}(2^{128})$) to recover H — and with H, forge any message. This is the "leaks H" of §3: the one-time pad must never repeat, which is exactly why nonce uniqueness is load-bearing.

4.6 Poly1305 is the same recipe

Poly1305 (§3) is the same construction in a prime field: a polynomial universal hash keyed by r, masked by a one-time pad s, mod $2^{130} - 5$. GHASH and Poly1305 are siblings — Wegman–Carter polynomial MACs; GHASH suits hardware (carry-less multiply), Poly1305 suits plain software.

4.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\mathrm{GF}(2^{128})$	field of 128-bit blocks: $+$ is XOR, $\times$ is carry-less multiply
H	the GHASH key, $H = E_k(0^{128})$
$X_i,\ Y_i$	input blocks (of $A \parallel C \parallel \mathrm{len}$) and the running value
$E_k(J_0)$	the one-time pad (J_0 = nonce-derived counter)
Wegman–Carter	universal hash $\oplus$ one-time pad = a MAC

5 Inside ChaCha20

§1 treated the stream cipher's keystream generator G as a black box, and §3 leaned on ChaCha20's *block counter* without saying what it is. Now we open the box. ChaCha20 (Bernstein, a refinement of Salsa20) is an *ARX* cipher: it builds its keystream with nothing but **A**dd, **R**otate, and **X**OR.

5.1 The state: a 4×4 grid of words

At heart is a *block function*: given the key, the nonce, and a block counter, it emits one 64-byte keystream block. It works on a state of 16 *words* (1 word = 32 bits), laid out as a 4×4 matrix — $16 \times 32 = 512$ bits = 64 bytes, exactly one block.

The four rows hold four different things:

c_0	c_1	c_2	c_3	4 constants ("`expand 32-byte k`")
k_0	k_1	k_2	k_3	256-bit key k (8 words)
k_4	k_5	k_6	k_7	
t	n_0	n_1	n_2	counter t | 96-bit nonce (3 words)

The top row is four fixed constants, the same in every ChaCha20 run — and not random-looking magic numbers. They are literally the 16 ASCII bytes of the text `expand 32-byte k` (a 16-character string = 16 bytes = 4 words of 32 bits). Cut the string into four 4-character pieces; each piece, read as a word, is one constant (its bytes packed little-endian, so `expa` becomes `0x61707865`):

	c_0	c_1	c_2	c_3
bytes	`expa`	`nd 3`	`2-by`	`te k`
word	`0x61707865`	`0x3320646e`	`0x79622d32`	`0x6b206574`

Why text? The constant is there as a fixed value that breaks the state's symmetry. And choosing it as plain readable text is a *nothing-up-my-sleeve* move: a carefully picked "magic" number could conceal a backdoor, but obvious ASCII — which even spells out that this is the 32-byte-key variant — leaves nowhere to hide one.

The counter t sits in word 12 — this is exactly the block counter of §3. (This is the RFC 8439 layout: a 32-bit counter and a 96-bit nonce.)

Three different time-scales share this one state. The key k is long-lived — reused across many messages. The nonce $n_0n_1n_2$ is drawn *fresh for each message* and held fixed while that message is encrypted. The counter t then runs $0, 1, 2, \ldots$ over the 64-byte blocks *within* that single message.

So within one message only t moves: bump it and the whole block changes, giving the next 64 bytes of keystream, while the nonce and key stay put. For the *next* message you draw a new nonce and reset t — the key usually does *not* change, yet the nonce changes anyway. Its freshness is not tied to the key; it must differ every message. That is the (key, nonce)-uniqueness rule of §1 and §3: never let a (key, nonce) pair repeat.

5.2 The quarter-round

All the mixing is done by one primitive, the *quarter-round* QR — a small function of *four* words. A call $\mathrm{QR}(a, b, c, d)$ takes four of the state's words and updates them in place. The names a, b, c, d are placeholders for "the four words this call happens to work on"; which of the state words $x_0, \ldots, x_{15}$ they stand for depends on where QR is applied — fixed by the column/diagonal pattern below. (They are generic argument names, *not* the constants $c_0 \ldots c_3$ from above.)

Its four words are updated by these eight steps, run *strictly in sequence* from top to bottom — each step reads the words the steps above it just wrote, so nothing here is parallel:

1. $a \leftarrow a \boxplus b$
2. $d \leftarrow (d \oplus a) \lll 16$
3. $c \leftarrow c \boxplus d$
4. $b \leftarrow (b \oplus c) \lll 12$
5. $a \leftarrow a \boxplus b$
6. $d \leftarrow (d \oplus a) \lll 8$
7. $c \leftarrow c \boxplus d$
8. $b \leftarrow (b \oplus c) \lll 7$

For example, step 2 reads the *new* a that step 1 just wrote, and step 3 the new d from step 2 — the updates chain.

Here $\boxplus$ is addition mod 2^{32}, $\oplus$ is XOR, and $w \lll k$ is a left-rotation of the 32-bit word w by k bits. That is the whole trio: add (mixes across bit positions via carries), rotate (moves bits between positions), XOR (the cheap, reversible combiner).

As a concrete binding: the first column quarter-round (below) sets $a = x_0,\ b = x_4,\ c = x_8,\ d = x_{12}$, while a diagonal one sets $a = x_0,\ b = x_5,\ c = x_{10},\ d = x_{15}$ — the same recipe, applied to different words.

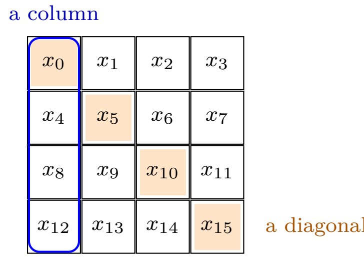
There are no S-boxes and no lookup tables — a deliberate choice we return to in §5 below.

The same quarter-round powers the BLAKE3 hash (in the hashing collection); BLAKE3's compression function is this mix run with right-rotations and two message words folded in.

5.3 Rounds: columns, then diagonals

One quarter-round touches only four words. To mix all 16, ChaCha alternates between the *columns* and the *diagonals* of the grid. A *double round* is a column round *followed by* a diagonal round (both, in that order — not a choice between them):

$$\begin{aligned} \text{column round:}\quad & \mathrm{QR}(x_0, x_4, x_8, x_{12}),\ \mathrm{QR}(x_1, x_5, x_9, x_{13}),\ \mathrm{QR}(x_2, x_6, x_{10}, x_{14}),\ \mathrm{QR}(x_3, x_7, x_{11}, x_{15}); \\ \text{diagonal round:}\quad & \mathrm{QR}(x_0, x_5, x_{10}, x_{15}),\ \mathrm{QR}(x_1, x_6, x_{11}, x_{12}),\ \mathrm{QR}(x_2, x_7, x_8, x_{13}),\ \mathrm{QR}(x_3, x_4, x_9, x_{14}). \end{aligned}$$



The name says the count: **ChaCha20** repeats this fixed pair 10 times — column, diagonal, column, diagonal, … — for 20 rounds in all. The order never varies, and nothing here is random.

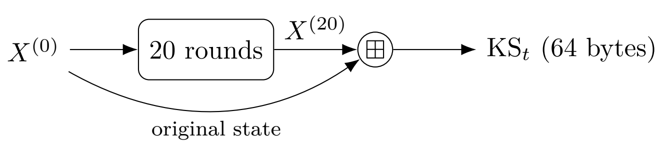
It *must* be deterministic: decryption regenerates the identical keystream from the same key, nonce, and counter (§1), so both sides have to walk exactly the same rounds. The only per-message freshness is the nonce, which is sent in the clear — not a secret random choice.

Diffusion compounds fast — after a couple of double rounds every output word depends on every input word.

5.4 Feed-forward, then the keystream

Call the starting matrix $X^{(0)}$ and the matrix after 20 rounds $X^{(20)}$. The block function does not output $X^{(20)}$ directly; it first adds the *original* state back in, word by word (mod 2^{32}):

$$\mathrm{KS}_t = X^{(20)} \boxplus X^{(0)}.$$



That feed-forward is what makes the block function one-way: the 20 rounds alone are a reversible permutation, but adding $X^{(0)}$ back means you cannot run it backwards to recover the key or counter from the keystream — the same trick as SHA-2's Davies–Meyer (in the hashing collection).

The result KS_t is the 64-byte keystream block for counter t. To encrypt, do what §1 and §2 described: generate $\mathrm{KS}_0, \mathrm{KS}_1, \ldots$ by incrementing t, and XOR them onto the plaintext.

This is the missing piece from §3: in ChaCha20-Poly1305 the counter-0 block supplies the Poly1305 key (its first 32 bytes) and the message rides on counters $1, 2, \ldots$.

5.5 Why ARX

The all-ARX design buys two things. Because every step is arithmetic on whole words — never a table indexed by secret data — ChaCha20 runs in *constant time*, immune to the cache-timing leaks that dog naive S-box ciphers.

And ARX is cheap and simple on any CPU, with no special hardware needed — which is precisely why ChaCha20-Poly1305 (§3) is the pick on phones and other machines without AES acceleration.

The 20 rounds are a generous margin: the best known attacks reach only about 7.

5.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
word	32 bits
$x_0, \dots, x_{15}$	the 16-word (4×4) ChaCha20 state
$c_0, \dots, c_3$	the four fixed constants (“`expand 32-byte k`”)
$k_0, \dots, k_7$	the eight words of the 256-bit key k
t	block counter (state word 12)
n_0, n_1, n_2	the three words of the 96-bit nonce
QR	the quarter-round (mixes four state words)
$\boxplus$	addition mod 2^{32} (word-wise)
$\lll k$	left-rotation of a 32-bit word by k bits
$X^{(0)}, X^{(20)}$	the state before / after the 20 rounds
KS_t	the 64-byte keystream block for counter t

6 Inside AES

§5 built a cipher from pure ARX. AES (Rijndael, the NIST standard since 2001) takes the other classic route: a *substitution–permutation network* (SP-network) — alternating a byte-substitution layer with a byte-shuffling layer, interleaved with key XORs.

Unlike ChaCha20, AES is a *block cipher* (§1): it maps one 128-bit block to one 128-bit block under the key, and to encrypt a real message you wrap it in a mode (§2).

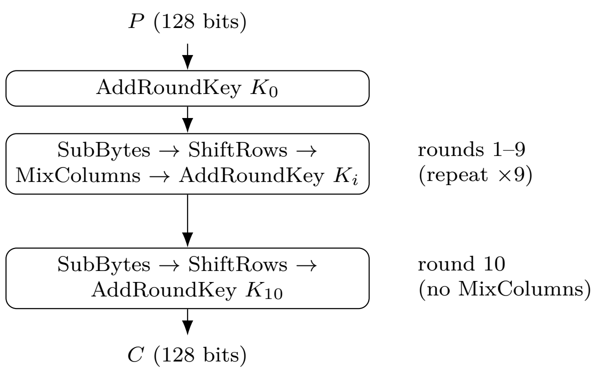
6.1 The state and the rounds

AES works on a *state*: the 128-bit block written as a 4×4 grid of *bytes* (16 bytes = 128 bits). Write $s_{r,c}$ for the byte in row r, column c ($r, c \in \{0, 1, 2, 3\}$). The input bytes fill the grid down columns — bytes 0–3 are column 0, bytes 4–7 column 1, and so on.

$s_{0,0}$	$s_{0,1}$	$s_{0,2}$	$s_{0,3}$
$s_{1,0}$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$
$s_{2,0}$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$
$s_{3,0}$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$

16 bytes, 4×4 (= 128 bits)

A 128-bit key gives $N_r = 10$ rounds (a 192-bit key gives 12, a 256-bit key 14). Each full round is four steps in fixed order — **SubBytes**, **ShiftRows**, **MixColumns**, **AddRoundKey** — with one twist at the ends:



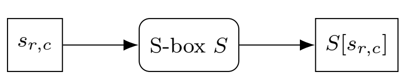
An extra AddRoundKey runs *before* round 1 (so no plaintext block is ever sent through the rounds unkeyed), and the *final* round drops MixColumns.

Three of the four steps — SubBytes, ShiftRows, MixColumns — are fixed, public, key-free shuffles. Only AddRoundKey injects the secret. The security comes from repeating the substitute-shuffle-mix-key alternation enough times that the two are hopelessly entangled.

6.2 SubBytes — the substitution (confusion)

Each byte is replaced by another via a fixed 256-entry lookup table, the *S-box S*:

$$s_{r,c} \leftarrow S[\, s_{r,c} \,] \qquad \text{for all } r, c.$$



This is the cipher's *only* nonlinear step, and it supplies *confusion* (Shannon's term): it makes the relationship between key and ciphertext intricate, so the output cannot be unpicked with linear algebra. The S-box itself is built from the multiplicative inverse in GF(2^8) followed by a fixed affine map — chosen for strong nonlinearity — where GF(2^8) is byte arithmetic (each byte a polynomial; add = XOR, multiply = carry-less multiply reduced by a fixed degree-8 polynomial).

6.3 ShiftRows — shuffle across columns (diffusion)

Row r is cyclically shifted left by r bytes: row 0 stays put, row 1 moves one place, row 2 two, row 3 three. (Cells show the byte index, numbered down columns.)

before					after				
0	4	8	12		0	4	8	12	
1	5	9	13	→	5	9	13	1	shift 1
2	6	10	14		10	14	2	6	shift 2
3	7	11	15		15	3	7	11	shift 3

On its own this just moves bytes around. Its job is to feed MixColumns: after the shift, each column holds bytes that started in four *different* columns, so the next step mixes bytes from all over the block — the start of *diffusion* (spreading each input's influence widely).

6.4 MixColumns — mix within a column (diffusion)

Each column is treated as a 4-byte vector and multiplied by a fixed matrix over GF(2^8):

$$\begin{pmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{pmatrix} = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{pmatrix} \qquad (c = 0, \dots, 3).$$

Spelling out the first row, with $\cdot$ a GF(2^8) multiply and $\oplus$ XOR:

$$s'_{0,c} = (2 \cdot s_{0,c}) \oplus (3 \cdot s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}.$$

The constants $1, 2, 3$ are just small field elements. ShiftRows and MixColumns together give full diffusion: after about two rounds every output byte depends on every input byte. (The final round omits MixColumns — at the very end it adds no security, and leaving it out keeps encryption and decryption structurally aligned.)

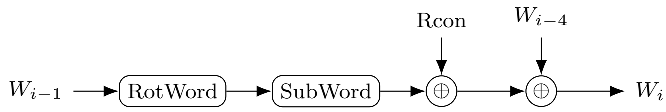
6.5 AddRoundKey and the key schedule

AddRoundKey XORs the round key K_i (also 128 bits, a 4×4 byte grid) into the state, byte for byte:

$$s_{r,c} \;\leftarrow\; s_{r,c} \oplus (K_i)_{r,c}.$$

This is the only step that touches the secret. XOR is its own inverse, so decryption removes the same key with another XOR.

The $N_r + 1 = 11$ round keys are not 11 independent secrets — they are stretched from the one key by the *key schedule*. Take the key as four 4-byte words $W_0, \dots, W_3$; the rest follow a recurrence. For most words $W_i = W_{i-4} \oplus W_{i-1}$; but every fourth word first passes W_{i-1} through a small function g (rotate its bytes, push them through the S-box, then XOR a round constant Rcon):



Round key K_i is the four words $W_{4i}, \ldots, W_{4i+3}$. Feeding the words through the S-box and the changing Rcon makes the round keys differ unpredictably, so learning one tells an attacker little about the others.

6.6 SP-network vs. ARX

AES and ChaCha20 (§5) reach the same goal by opposite means: AES leans on a lookup-table S-box plus linear mixing, ChaCha20 on add/rotate/XOR with no tables at all.

That S-box is the catch. Implemented naively as a table indexed by secret bytes, it can leak through cache-timing; this is exactly the weakness ChaCha20's table-free design avoids. AES's answer is hardware: the **AES-NI** instructions do a whole round at once, making AES both very fast *and* constant-time on modern CPUs.

That split is why §3's two AEADs coexist: AES-GCM wins where AES-NI is present, ChaCha20-Poly1305 where it is not.

	AES	ChaCha20
Type	block cipher	stream cipher
Design	SP-network (S-box)	ARX (no tables)
Unit	byte (GF(2^8))	32-bit word
Nonlinearity	the S-box	carry in $\boxplus$
Fastest with	AES-NI hardware	plain software

6.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$s_{r,c}$	state byte at row r, column c; the state is 4×4 bytes = 128 bits
N_r	number of rounds (10 for AES-128)
$S[\cdot]$	the S-box: a fixed 256-entry byte substitution
K_i	the i-th round key (128 bits), $i = 0, \ldots, N_r$
GF(2^8)	byte field: add = XOR, multiply = carry-less mod a fixed polynomial
W_i	key-schedule words (4 bytes each); $K_i = (W_{4i}, \ldots, W_{4i+3})$
RotWord, SubWord, Rcon	key-schedule helpers (rotate, S-box, round constant)
$\oplus$	bitwise XOR

7 Nonce management

§1, §2, and §3 all ended with the same warning: never reuse a (key, nonce) pair. This section is about how to actually *guarantee* that — and what to do when you cannot.

7.1 Why a repeated nonce is a disaster

Reuse breaks things in two separate ways.

Confidentiality. A repeated (key, nonce) gives the *same* keystream KS. Two messages masked with it satisfy

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

so the keystream cancels and the attacker learns $P \oplus P'$ — the two-time-pad break of §1.

Authenticity. In GCM (§3) a repeat is worse still: it lets the attacker solve for the GHASH key H and then *forge* tags for messages of their choosing. So a single slip turns into a total break, not just a leak.

That is why nonce uniqueness is load-bearing for the whole construction — and why it deserves real care.

7.2 Why $P \oplus P'$ is already a break

It is fair to object that $P \oplus P'$ looks useless: to recover P you would XOR by P', which you do not have. And that is right — *if the plaintexts were random*, $P \oplus P'$ would leak nothing. The break works because real plaintexts are *not* random: they are full of structure — known headers, fixed fields, ordinary words, predictable formatting.

That redundancy is the lever. Suppose you can *guess* a stretch of one message; call the guess g. XOR it into $P \oplus P'$:

$$(P \oplus P') \oplus g \;=\; P' \qquad \text{wherever } g = P.$$

A correct guess of part of P immediately hands you the same part of P' (and vice versa). This is the $P \oplus P' \oplus P' = P$ you spotted — except you supply a *guess* of one plaintext, not the unknown one.

Guesses come from *cribs*: a likely word or a known field. Slide a candidate — a common word like `the`, or a protocol header — along $P \oplus P'$ at each offset; a wrong position yields garbage in the other message, a right one yields readable text. Natural-language redundancy makes the right alignment stand out, and each recovered fragment becomes a crib for more. This is *crib dragging*, and it peels apart both messages.

Even with no crib to start, footholds are everywhere: $P \oplus P'$ is *zero* at every position where the two messages agree (a shared header or greeting appears as a run of zero bytes), and a space (`0x20`) XORed against a letter merely flips its case bit — a statistical tell that exposes where the spaces sit in each message.

So one keystream reuse downgrades "perfectly secret" to "two ciphertexts whose XOR is the XOR of two *predictable* messages" — routinely solvable. The one-time pad earns its name literally: it is secure only when the pad is used *once*.

7.3 Two ways to keep nonces unique

There are two strategies, with opposite failure modes.

Counter nonces (stateful): hold a counter and increment it for every message, never repeating a value. This gives the largest message budget — a 96-bit counter never wraps in practice. The danger is the *state*: a reboot, a restored backup, or a cloned VM can rewind the counter and silently reissue a value.

Random nonces (stateless): draw a fresh random nonce each time. There is no state to lose. The danger is *chance*: two messages may happen to draw the same nonce — the birthday problem, next.

7.4 The birthday bound on random nonces

With n-bit random nonces, after q messages under one key the chance that two coincide is about

$$\Pr[\text{collision}] \;\approx\; \frac{q^2}{2^{\,n+1}}.$$

To keep that negligible (say below 2^{-32}), you need $q \lesssim 2^{(n-32)/2}$. That ceiling is lower than people expect:

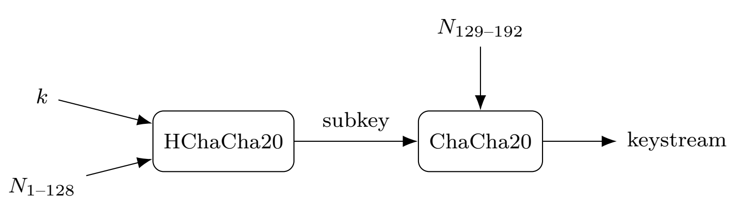
Nonce size	Used by	Random nonces safe up to
96 bits	GCM, ChaCha20 (§3)	$\approx 2^{32}$ messages
192 bits	XChaCha20 (below)	$\approx 2^{80}$ messages

So a 96-bit random nonce is fine for billions of messages but not for an unbounded stream; past the limit you must rekey — or use a bigger nonce.

7.5 Bigger nonces: XChaCha20

The clean fix for random nonces is to make the nonce so large that collisions never happen. XChaCha20 extends ChaCha20's 96-bit nonce to 192 bits.

It does so in two stages. First it runs *HChaCha20* — the ChaCha core (§5) used as a key-derivation step — on the key and the *first* 128 bits of the nonce, giving a fresh 256-bit subkey. Then it runs ordinary ChaCha20 under that subkey with the remaining 64 nonce bits.

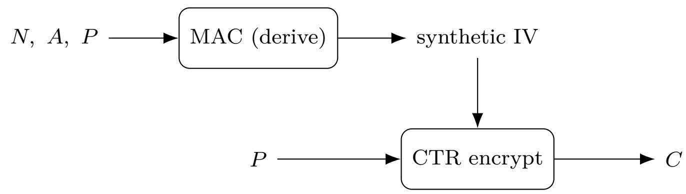


The upshot is a 192-bit nonce, whose birthday bound ($\approx 2^{80}$ messages) is so far off that you can pick nonces at random and never think about collisions again. (XSalsa20 does the same for Salsa20.)

7.6 Surviving reuse: misuse-resistant AEAD

A different philosophy: rather than *prevent* reuse, limit the damage when it happens. That is what AES-GCM-SIV does (SIV = *synthetic IV*).

Its trick is to derive the encryption IV from a MAC of the nonce, the associated data, *and the plaintext*, then CTR-encrypt with that IV (which also serves as the tag).



Because the IV depends on the message content, two *different* messages under the same $(\text{key}, \text{nonce})$ get different IVs — hence different keystreams, and no two-time pad. The only thing a repeat leaks is whether the *exact same* message was sent twice (identical ciphertext). That is a mild, bounded leak next to GCM's catastrophe.

The cost: the IV must be computed from the whole plaintext before encryption can start, so AES-GCM-SIV needs two passes and cannot encrypt a stream on the fly. You trade a little speed for robustness against a footgun.

7.7 Which to use

Situation	Use
Reliable counter state	a counter nonce (largest budget)
Stateless or many senders	random nonces — prefer 192-bit (XChaCha20)
Uniqueness can't be guaranteed	a misuse-resistant AEAD (AES-GCM-SIV)

Underneath all three, the rule never changes: a $(\text{key}, \text{nonce})$ pair must not repeat. When you are unsure whether it might, rekey.

7.8 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
N	the nonce
n	nonce length in bits
q	number of messages encrypted under one key
KS	keystream (§1)
P'	a second plaintext sent under the same (key, nonce)
g	a guessed plaintext fragment (a "crib")
subkey	the 256-bit ChaCha20 key HChaCha20 derives (XChaCha20)
synthetic IV	an IV derived from the message itself (AES-GCM-SIV)

8 Diffie–Hellman key exchange

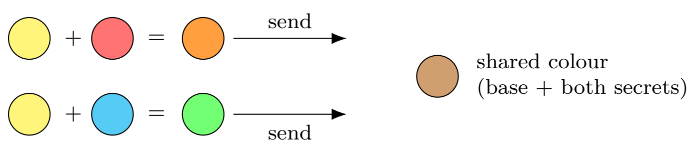
Every section so far opened with "a shared secret key k" and simply assumed both sides had it. But how do two parties who have never met agree on k over a channel an eavesdropper can read? You cannot just send k — the eavesdropper would see it.

Diffie–Hellman (1976) solves exactly this. It is the first piece of *public-key* crypto in this document: a way for two parties to agree on a shared secret in the open, where the eavesdropper sees *every* message exchanged and still cannot compute the secret.

8.1 The idea: mixing that will not un-mix

The trick is an operation that is easy to do but hard to undo. The classic intuition is *paint*: mixing two colours is easy, but separating a mixture back into its ingredients is practically impossible.

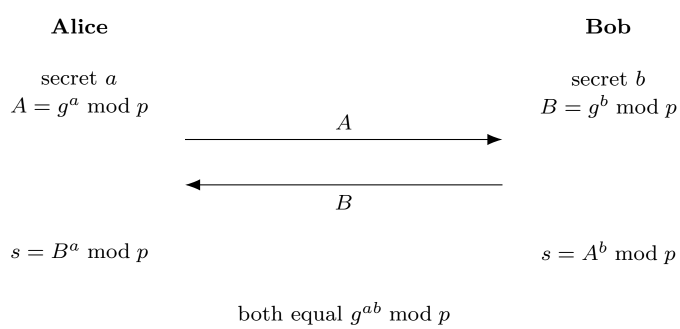
Both parties start from a public base colour. Each keeps a secret colour and mixes it into the base; they swap the resulting mixtures in the open. Each then stirs in their own secret again — and both arrive at the *same* final colour (base + both secrets), while an eavesdropper, holding only the two public mixtures, cannot un-mix them to reach it.



The mathematical "mixing" is *modular exponentiation*. Fix a large public prime p and a public base g. Raising g to a power mod p is easy; but recovering the exponent from the result — the *discrete logarithm* — is believed infeasible for large p. That one-wayness is what the eavesdropper runs into.

8.2 The exchange

Public, known to everyone: the prime p and the base g. Each party draws one private exponent.



Both land on the same value because the exponents just multiply:

$$B^a = (g^b)^a = g^{ab} = (g^a)^b = A^b \pmod{p}.$$

The eavesdropper sees g, p, $A = g^a$, and $B = g^b$, but to get g^{ab} they would need a or b — i.e. solve a discrete logarithm. That is the *Diffie–Hellman problem*, believed hard for well-chosen large p (today, 2048 bits or more).

Notation flag. The modulus here is the lower-case prime p, *not* the plaintext P used elsewhere in this document.

8.3 A worked example

Small numbers make it concrete. Take public $p = 23$, $g = 5$.

- Alice picks $a = 6$ and sends $A = 5^6 \bmod 23 = 8$.
- Bob picks $b = 15$ and sends $B = 5^{15} \bmod 23 = 19$.
- Alice computes $s = B^a = 19^6 \bmod 23 = 2$.
- Bob computes $s = A^b = 8^{15} \bmod 23 = 2$.

Both get $s = 2$, which equals $g^{ab} = 5^{90} \bmod 23 = 2$. An eavesdropper holds $g = 5,\ p = 23,\ A = 8,\ B = 19$ — and to find a must solve $5^a \equiv 8 \pmod{23}$ by discrete log. At 23 that is a quick search; at 2048 bits it is hopeless.

8.4 From shared secret to key

The shared s is a big number, not yet a key. You do not use it directly: run it through a key-derivation function (a hash-based KDF, from the hashing collection) to get a uniform symmetric key

$$k = \mathrm{KDF}(s).$$

From there, everything in §1–§7 applies — k feeds AES-GCM or ChaCha20-Poly1305 as usual. Diffie–Hellman is the missing first step that produces the k the rest of the document assumed.

8.5 The catch: it does not authenticate

Plain Diffie–Hellman defeats a *passive* eavesdropper, but not an *active* attacker. Nothing in the exchange proves *who* is on the other end.

A *man-in-the-middle*, Mallory, can run a separate exchange with each side — agreeing on a secret s_1 with Alice and a different s_2 with Bob — then sit in the middle, decrypting with one key and re-encrypting with the other, reading everything. Neither side notices.

$$\boxed{\text{Alice}} \xleftrightarrow{\text{DH} \to s_1} \boxed{\text{Mallory}} \xleftrightarrow{\text{DH} \to s_2} \boxed{\text{Bob}}$$

So the exchange must be *authenticated*: the public values are tied to an identity by a signature, a certificate, or a pre-shared key. TLS, for instance, has the server sign its Diffie–Hellman value. Diffie–Hellman gives secrecy of the agreement; proving who you agreed *with* is a separate, mandatory job.

8.6 In practice: ECDH and forward secrecy

Two refinements dominate real use.

Elliptic-curve Diffie–Hellman (ECDH) runs the same idea in a different group: points on an elliptic curve, where "raise to a power" becomes "multiply a point by a scalar." The hard problem is the same shape, but breaking it needs far bigger effort per bit — so a 256-bit curve key (e.g. X25519) matches a 3072-bit classical one. Smaller and faster, it is the modern default.

Forward secrecy comes from using *ephemeral* secrets — fresh a, b per session, discarded afterwards (the "E" in ECDHE). Then even if a party's long-term key is later stolen, past sessions stay secret: their one-time exponents are already gone.

8.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
p	large public prime, the modulus (*not* the plaintext P)
g	public base (generator)
$a,\ b$	Alice's and Bob's private exponents (secret)
$A,\ B$	public values $g^a \bmod p$ and $g^b \bmod p$
s	the shared secret $g^{ab} \bmod p$
KDF	key-derivation function turning s into the key k

9 Digital signatures

§8 left a hole: Diffie–Hellman agrees a key but cannot prove *who* is on the other end, so a man-in-the-middle slips in. Closing that hole is what digital signatures do.

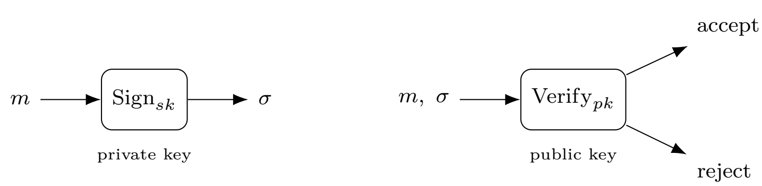
Could a MAC (§3) prove identity? No — a MAC needs a *shared* secret, and a shared secret is exactly what we have not got yet. We need to prove a message came from a specific party with *no* pre-shared secret, in a way *anyone* can check. That is a signature.

9.1 Key pairs: sign with private, verify with public

Each party holds a *key pair*: a *private key sk* kept secret, and a *public key pk* published to the world. They drive two operations:

$$\sigma = \mathrm{Sign}_{sk}(m), \qquad \mathrm{Verify}_{pk}(m, \sigma) \in \{\mathrm{accept}, \mathrm{reject}\}.$$

Here m is the message and σ its signature.



Only the holder of sk can produce a σ that verifies, and pk reveals nothing that helps forge one. So *anyone* can verify, but only *one* party can sign.

That asymmetry is the whole difference from a MAC:

	MAC (§3)	Signature
Keys	one shared secret	private (sign) + public (verify)
Who can sign	both parties	only the private-key holder
Who can verify	both parties	anyone with pk
Shared secret	required	none
Non-repudiation	no	yes

The last row matters: because only the signer holds sk, they cannot later deny a valid signature — *non-repudiation*, something a MAC (where both sides share the key) can never give.

9.2 Sign the hash, not the message

The signature math takes a fixed-size input, but messages are any length. So you hash first (*hash-then-sign*):

$$\sigma = \mathrm{Sign}_{sk}\big(H(m)\big),$$

where H is a collision-resistant hash (from the hashing collection). Collision resistance is essential: if $H(m) = H(m')$, one signature would be valid for both messages.

9.3 How it works: RSA and elliptic curves

Every scheme rests on a *trapdoor*: signing is easy with sk, forging is infeasible without it.

RSA (trapdoor: factoring is hard). The public key is a modulus $N = pq$ (product of two big primes) and an exponent e; the private key is d with $ed \equiv 1 \pmod{\varphi(N)}$. Signing applies the private exponent, verifying the public one, and they undo each other:

$$\sigma = H(m)^d \bmod N, \qquad \text{accept iff } \sigma^e \equiv H(m) \pmod{N}.$$

Forging needs d, which needs factoring N — infeasible for large N. (Real RSA pads the hash first, e.g. PSS; signing the bare hash is unsafe.)

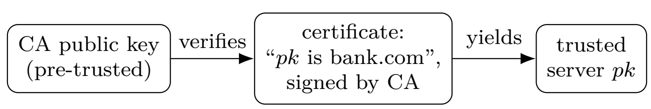
Elliptic curves (trapdoor: the curve discrete log of §8). The private key is a secret scalar, the public key a curve point. ECDSA and Ed25519 are the modern schemes — much smaller and faster than RSA for equal security.

A nonce footgun, again. ECDSA needs a fresh, unpredictable secret value for *each* signature. Reuse it — or let it be guessable — across two signatures and the private key falls straight out, the same nonce-reuse disaster as §7. It has broken real systems (the Sony PS3, early Bitcoin wallets). Ed25519 removes the footgun by deriving that value deterministically from the key and message.

9.4 Certificates: trusting a public key

A signature proves "whoever holds *sk* signed this" — but how do you know a given *pk* belongs to the *right* party (the real bank, not Mallory)?

A *certificate* answers that: a trusted *Certificate Authority* (CA) signs a statement binding a public key to an identity. You check that certificate with the CA's *own* public key, which you already trust because it ships with your OS or browser.



This is the *chain of trust*: trust in one pre-installed CA key extends, via signatures, to the public key of any site it vouches for.

9.5 Authenticating Diffie–Hellman

Now §8's man-in-the-middle is finished. Each party *signs* its Diffie–Hellman public value with its private key; the other verifies the signature with the partner's certificate-vouched public key.

Mallory cannot forge that signature, so she cannot substitute her own Diffie–Hellman value without being caught — the exchange is now *authenticated*. Key agreement (§8) plus signed identities is the combination every real secure channel is built from.

9.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$sk,\ pk$	private (signing) key and public (verification) key
$\mathrm{Sign},\ \mathrm{Verify}$	the signing and verification operations
σ	the signature
$m,\ H(m)$	the message and its hash (what actually gets signed)
$N,\ e,\ d$	RSA modulus $N = pq$, public exponent e, private exponent d
CA	Certificate Authority: vouches for a public key

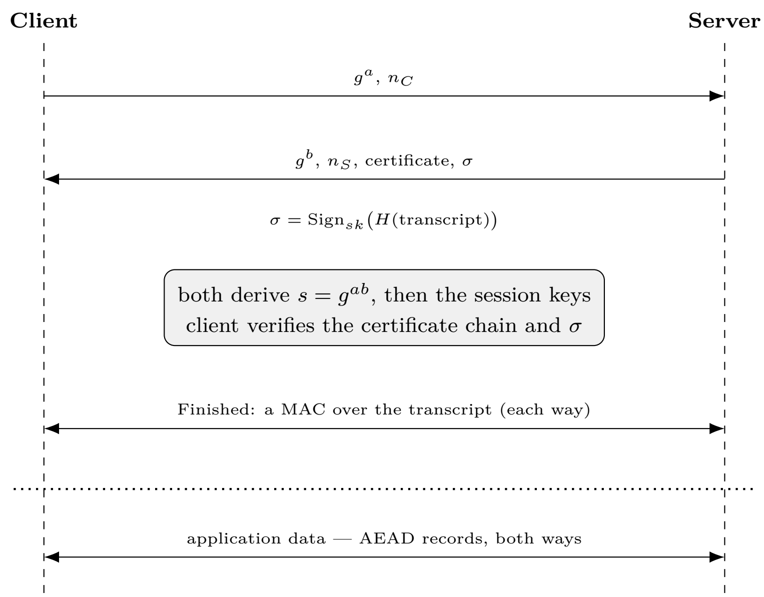
10 Putting it together: a secure channel

Every piece is now on the table:

- AEAD for confidentiality *and* integrity (§3), built on AES (§6) or ChaCha20 (§5);
- Diffie–Hellman to agree a key over a public channel (§8);
- signatures and certificates to prove identity (§9);
- nonce discipline to keep it all safe (§7).

A *secure channel* — TLS is the everyday example — bolts them into one protocol. Here is a simplified TLS-1.3-style flow.

10.1 The handshake



The client opens with an *ephemeral* Diffie–Hellman share g^a (§8) and a fresh random nonce n_C. "Ephemeral" means a is generated for this session and discarded after — that is what buys forward secrecy.

The server replies with its own share g^b, its *own* nonce n_S, its certificate (§9), and a signature σ. The two nonces are *different* — one chosen by each side — and both are folded into the key derivation, so the session keys draw on randomness from both parties (and a replayed handshake yields different keys).

What is that signature? Not encryption. It is

$$\sigma = \mathrm{Sign}_{sk}\big(H(\text{transcript})\big) :$$

the server signs a hash of the *entire handshake transcript so far* (both shares, both nonces, the certificate) with its *private* signing key (§9), and the client checks it with the public key in the certificate. Signing the whole transcript — not just g^b — binds the server's identity to *this* exact exchange, so a man-in-the-middle cannot lift the signature onto different values.

Both sides now compute the shared secret $s = g^{ab}$ (§8). Crucially, the client verifies the certificate chain (up to a trusted CA) and the signature *before* trusting s: Mallory cannot forge σ, so she cannot substitute her own g^b — the man-in-the-middle is shut out.

Finally each side sends a *Finished* message. It is not a signature but a MAC (§3): the KDF derives one more value from s — a separate *finished key* k_{fin} — and each side computes

$$\text{Finished} = \text{MAC}_{k_{\text{fin}}}\big(H(\text{transcript})\big),$$

an HMAC over the hash of all handshake messages so far. Each verifies the other's. A match proves two things at once: both sides derived the same secret (only a holder of k_{fin} could produce the tag) and both saw the identical, untampered handshake (any altered message changes $H(\text{transcript})$). Note the division of labour — the signature σ used the server's *private* key to prove *identity*; the Finished uses a *shared derived* key to prove *agreement*. Only then does application data flow.

10.2 Deriving the session keys

The raw s is never used directly. It is fed — with both nonces and the transcript — into a key-derivation function (HKDF, from the hashing collection), which emits *two different* keys, one for each direction:

$$\underbrace{k_{C\to S}}_{\text{client}\to\text{server}},\ \underbrace{k_{S\to C}}_{\text{server}\to\text{client}} = \text{KDF}(s,\ n_C,\ n_S,\ \text{transcript}), \qquad k_{C\to S} \neq k_{S\to C}.$$

$$s,\ n_C,\ n_S \longrightarrow \boxed{\text{HKDF}} \longrightarrow \begin{matrix} k_{C\to S} \\ k_{S\to C} \end{matrix}$$

They are *not* the same key. Why two? If both directions shared one key and each began its counter nonce (§7) at zero, the client's first record and the server's first record would reuse the same (key, nonce) pair — the two-time-pad disaster of §7. A separate key per direction removes that risk entirely.

But *both* parties derive the same pair from s, so each holds *both* keys — a key names a *direction*, not an owner. The client encrypts its traffic under $k_{C\to S}$ and the server decrypts it under that *same* $k_{C\to S}$; the server's replies travel under $k_{S\to C}$, which the client uses to open them. AEAD is symmetric (§3) — the same key both seals and opens — so two shared keys are all it takes.

10.3 The data phase

From here every message — every *record* — is sealed with AEAD (§3) under the direction's key, using AES-GCM or ChaCha20-Poly1305.

Each record carries a counter nonce (§7) that increments per record: it never repeats, and it doubles as a sequence number, so a dropped, reordered, or replayed record is caught.

Record headers ride along as *associated data* (§3) — authenticated so they cannot be tampered with, but left readable because routing needs them.

10.4 Where each guarantee comes from

The point of the assembly is that each property of the channel traces to exactly one piece:

Guarantee	Comes from	Where
Confidentiality	AEAD encryption	§3, §6, §5
Integrity & authenticity (data)	the AEAD tag	§3
Shared key over a public channel	(ephemeral) Diffie–Hellman	§8
Identity / no man-in-the-middle	signatures + certificates	§9
Forward secrecy	ephemeral DH keys	§8
No nonce reuse	per-direction keys, counter nonces	§7

No single primitive is a secure channel on its own; the channel is what you get when they are composed in the right order, each covering the gap the previous one left.

10.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
g^a, g^b	the client's and server's ephemeral DH shares (§8)
s	the shared secret g^{ab} (§8)
$n_C,\ n_S$	the client's and server's (distinct) handshake nonces
σ	the server's signature, $\mathrm{Sign}_{sk}(H(\text{transcript}))$
transcript	all handshake messages exchanged so far
$k_{C\to S},\ k_{S\to C}$	the two *distinct* per-direction session keys
k_{fin}	a separate finished key (from s) that keys the Finished MAC
record	one AEAD-sealed message in the data phase

11 A post-quantum note

A large-scale quantum computer would not break cryptography evenly. The damage is lopsided — and which half falls is exactly the split this document has been built around.

11.1 What a quantum computer breaks

Shor's algorithm (§24) solves integer factoring and discrete logarithms efficiently on a quantum computer. Those are precisely the hard problems underneath the public-key half: factoring for RSA (§9), discrete log for Diffie–Hellman and ECDH (§8), and the curve discrete log for ECDSA/Ed25519 (§9). All of it — key agreement *and* signatures, hence the whole handshake of §10 — would fall completely.

Grover's algorithm (§25) only speeds up brute-force search quadratically (square-root). Against the symmetric world — AES (§6), ChaCha20 (§5), AEAD, hashes — it merely halves the effective strength, which you fix by doubling the key or output size. The symmetric core survives essentially intact.

	Quantum attack	Effect	Fix
Symmetric (AES, ChaCha20)	Grover	strength halved	bigger keys (AES-256)
Public-key (RSA, DH, ECDSA)	Shor	broken	new math (below)

11.2 Harvest now, decrypt later

The machine is years off, but for *confidentiality* the threat is already present. An adversary can record your encrypted traffic today and store it until a quantum computer exists, then go back and break the key exchange that protected it. So anything that must stay secret for a decade needs post-quantum key agreement *now*.

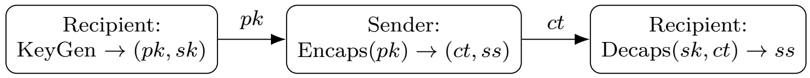
Authentication is less urgent. You cannot forge a signature on a past message after the fact, so post-quantum signatures only need to be in place by the time quantum computers actually arrive — not before. NIST's signature standard is ML-DSA (§29).

11.3 The replacements

Post-quantum cryptography rebuilds key agreement and signatures on problems believed hard even for quantum computers — mostly *lattice* problems (finding short vectors in high-dimensional lattices, e.g. Learning With Errors; detailed in §27), for which no efficient quantum algorithm is known. NIST standardised three in 2024:

Classical (broken)	Post-quantum replacement	Based on
(EC)DH key agreement (§8)	ML-KEM / Kyber (FIPS 203)	lattices (LWE)
RSA, ECDSA signatures (§9)	ML-DSA / Dilithium (FIPS 204)	lattices
(a conservative backup)	SLH-DSA / SPHINCS$^+$ (§30)	hash functions only

The new key agreement is shaped differently from Diffie–Hellman: it is a *KEM* (key encapsulation mechanism), not a mutual exchange of shares. The recipient publishes a public key; the sender *encapsulates* a random shared secret to it, producing a ciphertext; the recipient *decapsulates* that ciphertext with the private key to recover the same secret. NIST's standard of this shape is ML-KEM, built in full in §28.

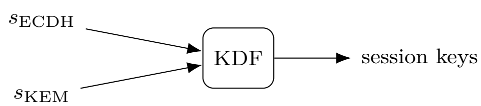


Both sides end up with the shared secret ss, which feeds a KDF exactly as Diffie–Hellman's s did (§8) — same outcome, different machinery. (These schemes lean on SHAKE/Keccak internally — the same sponge from the hashing collection.)

11.4 Hybrid, for now

Deployments today do not yet bet the channel on the young lattice schemes alone. They go *hybrid*: run a classical ECDH (§8) *and* a post-quantum KEM together, and feed *both* shared secrets into the KDF:

$$k = \mathrm{KDF}\big(s_{\mathrm{ECDH}} \parallel s_{\mathrm{KEM}}\big).$$



You are safe if *either* holds: the post-quantum part defeats harvest-now-decrypt -later, while the battle-tested ECDH part guards against a flaw later found in the new scheme.

11.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
KEM	key encapsulation mechanism (post-quantum key agreement)
$pk,\ sk$	the recipient's public and private KEM keys
$ct,\ ss$	the encapsulated ciphertext and the shared secret it carries
$s_{\mathrm{ECDH}},\ s_{\mathrm{KEM}}$	the classical and post-quantum shared secrets (hybrid)

12 Randomness

This document has quietly rested on two words, over and over: "pick at random." A random key (§1), a fresh random nonce (§2, §7), random Diffie–Hellman secrets a, b (§8), a fresh per-signature value (§9). All of that security assumes those values are *unpredictable*.

If an attacker can predict or even narrow down your "random" choice, they recover the key or secret directly — without touching AES, without factoring, without any of the hard problems. Bad randomness is a side door around all the strong math. So it is worth asking where the randomness actually comes from.

12.1 Random means unpredictable

Cryptographic randomness is not about *statistical* prettiness — passing dice-roll uniformity tests. It is about *unpredictability*: given everything an attacker knows, including all previous outputs, they must not guess the next bit better than chance.

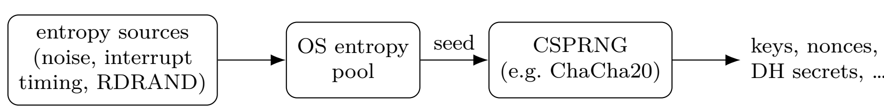
A sequence can be statistically flawless yet completely predictable — the digits of π, or the output of a simple linear congruential generator — and is then worthless for keys. "Looks random" is not "is random."

12.2 Entropy plus a CSPRNG

Real systems get there with two ingredients.

Entropy is genuine physical unpredictability the machine harvests: thermal and electrical noise, the jitter in the timing of interrupts (keystrokes, disk, network), or a dedicated hardware generator (Intel's RDRAND). It is truly unpredictable, but slow and scarce.

A *CSPRNG* (cryptographically secure pseudo-random number generator) is a deterministic algorithm that stretches a small high-entropy *seed* into an endless stream of unpredictable-looking bits, built from a crypto primitive. Predicting its output without the seed would mean breaking that primitive.



The operating system wires this up for you: it gathers entropy, seeds a CSPRNG, and serves the bits through a system call — `getrandom()` or `/dev/urandom` on Linux, `BCryptGenRandom` on Windows, `SecRandomCopyBytes` on macOS. Draw every key, nonce, and secret from there.

A CSPRNG is, in fact, essentially a stream cipher run for its keystream: the seed plays the role of key-plus-nonce, and the output is the keystream. ChaCha20 (§5) is exactly this shape — and Linux's `/dev/urandom` is built on ChaCha20. The same ARX engine that encrypts also manufactures randomness.

12.3 When randomness fails

Strong cipher, broken generator is one of the worst failure modes there is — and it keeps happening.

Debian OpenSSL (2008). A cleanup removed the line that mixed entropy into the seed, leaving essentially just the process ID — about 2^{15} possibilities. Every key Debian generated for roughly two years was one of ∼32,000, all enumerable. The ciphers were fine; the randomness was not.

ECDSA nonces (Sony PS3, 2010*; some Bitcoin wallets).* This is §9's footgun in the wild: Sony used a *constant* value where a fresh random one was required, and weak wallet generators produced repeated or biased nonces. Either way the private key fell straight out — a randomness failure, not a flaw in the signature scheme.

Dual_EC_DRBG (∼2007*).* A NIST-standardised CSPRNG whose elliptic-curve constants could be chosen so that whoever knew a secret relationship could predict all of its output — a suspected backdoor. It is the mirror image of §5's nothing-up-my-sleeve constants: a generator is only as trustworthy as the provenance of its magic numbers.

12.4 In practice

Never roll your own, and never use a *non-cryptographic* PRNG — C's `rand()`, a Mersenne Twister, a linear congruential generator — for anything secret. They are built for speed and uniformity, not unpredictability, and are trivially predicted from a few outputs.

Use the operating system's CSPRNG (`getrandom`, `/dev/urandom`, `BCryptGenRandom`), or a vetted library that wraps it.

Watch the low-entropy moments: a device at first boot, a freshly imaged embedded board, or a just-cloned virtual machine may have gathered little entropy and hand out weak — or duplicated — randomness. A cloned VM that copies the CSPRNG state will reissue the very same "random" nonces, which is the clone hazard already flagged in §7.

12.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
entropy	genuine physical unpredictability the machine harvests
CSPRNG	cryptographically secure generator: seed → unpredictable stream
seed	the high-entropy value that initialises the CSPRNG

13 Public-key encryption

We have met two public-key tools: key agreement (Diffie–Hellman, §8) and signatures (§9). The third is *public-key encryption* — scrambling a message so that only the holder of a private key can read it.

13.1 The third public-key tool

Each recipient has a key pair: a *public key pk* anyone may use to encrypt, and a *private key sk* only they hold to decrypt.

$$c = \mathrm{Enc}_{pk}(m), \qquad m = \mathrm{Dec}_{sk}(c).$$

Anyone with *pk* can encrypt to the recipient; only *sk* can recover the message.

This is the mirror of signatures (§9). Signing uses the *private* key and verifying the *public* one; encryption is the other way around — the *public* key encrypts, the *private* key decrypts. (A party usually keeps *separate* pairs for signing and for encryption.)

Unlike Diffie–Hellman, it is one-shot and non-interactive: there is no handshake and the recipient need not be online. You encrypt to their published *pk* whenever you like.

13.2 Textbook RSA, and why it needs padding

RSA can encrypt directly, reusing the modulus $N = pq$ and exponents of §9 — but with the roles swapped from signing:

$$c = m^e \bmod N \quad (\text{public } e,\ \text{anyone}), \qquad m = c^d \bmod N \quad (\text{private } d).$$

The factoring trapdoor protects it, just as for signatures.

But raw “textbook” RSA is unsafe. It is deterministic — the same message always gives the same ciphertext, leaking equality, and a small or guessable message space can simply be tried — and it is malleable. Real RSA encryption first pads the message with randomness (*RSA-OAEP*), exactly as signing uses PSS. Never encrypt with bare RSA.

13.3 Why you don’t encrypt the data directly

Two hard limits stop you from public-key-encrypting a whole message.

Public-key operations are *slow* — orders of magnitude slower than AES (§6) or ChaCha20 (§5).

And they are *size-limited*: RSA can only encrypt a message smaller than its modulus (RSA-2048 tops out around 256 bytes). You cannot wrap a large file this way at all.

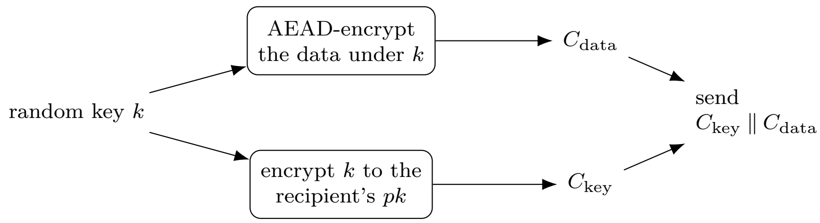
So public-key encryption is used to protect one small thing — a key — and symmetric crypto carries the bulk. That is *hybrid encryption*.

13.4 Hybrid encryption

The recipe:

1. generate a fresh random symmetric key k (a per-message “content key”);
2. encrypt the data with k using a fast AEAD (§3) — call it C_{data};

3. encrypt k to the recipient's public key — call it C_{key};
4. send $C_{\text{key}} \| C_{\text{data}}$.



The recipient runs $\text{Dec}_{sk}(C_{\text{key}})$ to recover k, then AEAD-decrypts C_{data} with it.

The payoff is the best of both worlds: public-key *convenience* (encrypt to anyone's pk, with no prior shared secret and no handshake) with symmetric *speed* (the bulk data goes through AEAD). It is how PGP/GPG, age, S/MIME, and libsodium sealed boxes encrypt files and mail — and it is the data phase of TLS once the handshake of §10 has settled on a key.

13.5 The modern shape: KEM + DEM

Today the pattern is named in two halves:

- a **KEM** (key encapsulation mechanism) delivers the symmetric key — classic RSA-style "pick k and encrypt it" is one KEM, while the cleaner modern KEMs (including the post-quantum ML-KEM of §11) instead $\text{Encaps}(pk) \to (ct, ss)$ and let the recipient $\text{Decaps}(sk, ct) \to ss$, then derive the key with a KDF;
- a **DEM** (data encapsulation mechanism) is just the AEAD that seals the data under that key.

So hybrid encryption is exactly KEM + DEM — the same encapsulation idea as §11, applied here to encrypt to a public key rather than to run an interactive exchange.

13.6 Who sent it?

Encryption hides the message but proves nothing about its origin — *anyone* can encrypt to your public key. To authenticate the sender you add a signature (§9) over the message or ciphertext. (An anonymous "sealed box" gives no sender authentication; an authenticated box binds a specific sender.) Confidentiality and authenticity remain separate goals, exactly as in §3.

13.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$pk,\ sk$	recipient's public / private keys (separate from signing)
$\text{Enc}_{pk},\ \text{Dec}_{sk}$	public-key encrypt / decrypt
$C_{\text{key}},\ C_{\text{data}}$	PK-encrypted content key; AEAD-encrypted data
KEM, DEM	key- / data-encapsulation mechanism (two halves of hybrid)
$N,\ e,\ d$	RSA modulus, public and private exponents (§9)

14 Inside elliptic curves (ECDH)

Diffie–Hellman (§8) and signatures (§9) work in *any* group where the discrete logarithm is hard. We have kept pointing at elliptic curves as "the same idea in a different group, with smaller keys." This section opens that group up.

14.1 Why elliptic curves

Classical Diffie–Hellman lives in the integers mod a prime and needs ∼3072-bit keys. An elliptic curve gives a group where the discrete log is far harder *per bit*, so a 256-bit key matches a 3072-bit classical one — smaller and faster. That is why curves (X25519 in §8, Ed25519 in §9) are the modern default.

14.2 Adding points: the group

An elliptic curve over a finite field is the set of points (x, y) — with x, y in $\mathbb{F}_p$ (integers mod a prime p) — satisfying

$$y^2 = x^3 + \alpha x + \beta,$$

together with one extra "point at infinity" $\mathcal{O}$.

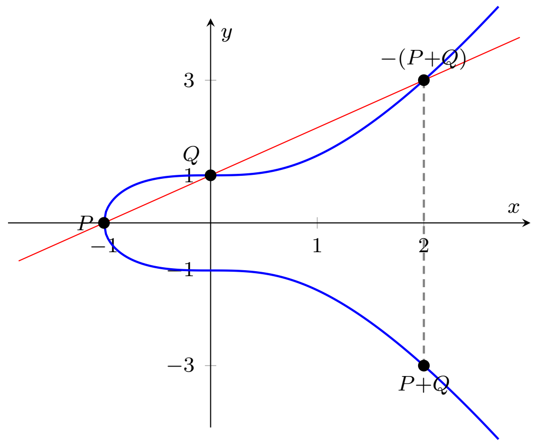
Because the equation contains y^2, whenever (x, y) lies on the curve so does $(x, -y)$ — the curve is symmetric about the x-axis. That is precisely why negation is a reflection across that axis, $-(x, y) = (x, -y)$. (This holds for the short Weierstrass form used over a prime field; the Edwards form of §18 is instead symmetric about the y-axis, with $-(x, y) = (-x, y)$.)

These points form a group under a geometric *chord-and-tangent* rule:

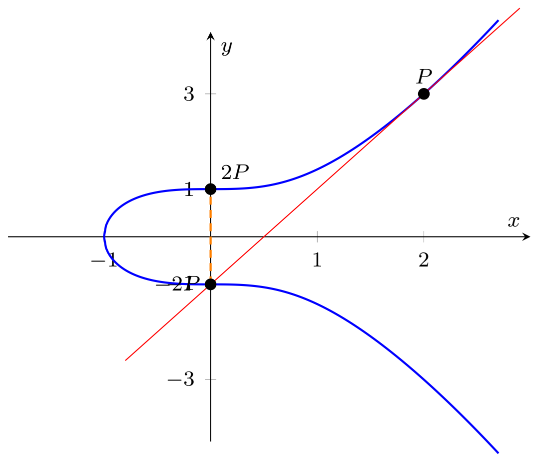
- **Add** $P + Q$ (distinct points): draw the straight line through P and Q; it meets the curve at exactly one more point; reflect that across the x-axis — the reflection is $P + Q$.
- **Double** $2P = P + P$: use the *tangent* at P instead of a chord; it meets the curve once more; reflect to get $2P$.
- $\mathcal{O}$ is the identity ($P + \mathcal{O} = P$), and the reflection of P across the x-axis is its inverse $-P$ (so $P + (-P) = \mathcal{O}$).

To make the rule concrete, take the curve $y^2 = x^3 + 1$ over the reals.

Addition. With $P = (-1, 0)$ and $Q = (0, 1)$, the line through them is $y = x + 1$; it meets the curve again at $(2, 3)$, and reflecting that across the x-axis gives $P + Q = (2, -3)$.



Doubling. With $P = (2, 3)$, the *tangent* there is $y = 2x - 1$; it meets the curve again at $(0, -1)$, and reflecting gives $2P = (0, 1)$.



The result is an abelian group — exactly the structure §8 and §9 need. (Over $\mathbb{F}_p$ the "geometry" is really the same operations done with modular arithmetic, but the chord-and-tangent rule is the picture to keep in mind.)

14.3 Scalar multiplication: the one-way step

Adding P to itself k times gives *scalar multiplication*

$$kP = \underbrace{P + P + \cdots + P}_{k \text{ times}},$$

the curve's analogue of the exponentiation g^k of §8.

Each $+$ in that sum is the chord-and-tangent addition defined above: $2P$ is a tangent (doubling) step, $3P = 2P + P$ a chord step, and so on. So scalar multiplication is nothing but that one geometric operation, repeated.

Over a finite field the multiples of a point eventually cycle back: the smallest n with $nG = \mathcal{O}$ is the *order* of G, and its n distinct multiples $G, 2G, \ldots, (n-1)G, \mathcal{O}$ form the cyclic group we compute in. Scalars are therefore taken mod n; a *prime* order n is preferred, as it leaves no smaller subgroups for an attacker to exploit.

$$G \xrightarrow{+G} 2G \xrightarrow{+G} 3G \longrightarrow \cdots \longrightarrow kG$$

Forward is cheap: *double-and-add* (the additive twin of square-and-multiply) computes kP in about $\log k$ steps, so kP is fast even for an enormous k. Backward is hard: given P and kP, recovering k is the *elliptic-curve discrete logarithm problem* (ECDLP). The best classical attacks cost about $\sqrt{\text{group size}}$, so a 256-bit curve gives ∼128-bit security — the reason the keys are so small.

14.4 ECDH

Elliptic-curve Diffie–Hellman is just §8's protocol run in this group. Public to all: a curve and a base point G.

$$\begin{array}{ccc}
\textbf{Alice} & & \textbf{Bob} \\
\text{secret scalar } a & & \text{secret scalar } b \\
A = aG & & B = bG \\
 & \xrightarrow{\quad A \quad} & \\
 & \xleftarrow[\quad B \quad]{} & \\
S = aB & & S = bA \\
 & \text{both equal } abG &
\end{array}$$

Both land on the same point because the scalars commute: $aB = a(bG) = abG = b(aG) = bA$. An eavesdropper sees G, aG, bG, but reaching abG needs a or b — the ECDLP. The shared secret is a point; you take its x-coordinate and run it through a KDF to get the key, exactly as $s \to k$ in §8.

It is structurally identical to classical DH; only the group changed:

	Classical DH (§8)	ECDH
Group	integers mod p	points on a curve
Mixing	$g^a \bmod p$	aG (scalar mult)
Secret	exponent a	scalar a
Public	g^a	aG
Shared	g^{ab}	abG
Hard problem	discrete log	ECDLP
Key size (∼128-bit)	∼3072-bit	∼256-bit

Notation flag. Here p is the field prime (as in §8), and P, Q, R denote curve *points* — not the plaintext P of earlier sections; the curve's constants are α, β.

14.5 Curve25519 / X25519 in practice

The dominant instantiation is *Curve25519*, a specific curve over the prime $p = 2^{255} - 19$, used by the *X25519* function (ECDH on that curve).

It was chosen for speed and for safety: its parameters are rigid and transparently derived (nothing-up-my-sleeve, like the constants of §5 and §12), and it avoids several common implementation pitfalls.

X25519 is the modern default — TLS, SSH, Signal, WireGuard — and its Edwards-curve cousin powers the Ed25519 signatures of §9.

14.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\mathbb{F}_p$	the finite field of integers mod a prime p
$\alpha,\ \beta$	the curve's constants in $y^2 = x^3 + \alpha x + \beta$
P, Q, R	curve *points* (not the plaintext of earlier sections)
$\mathcal{O}$	the point at infinity — the group identity
G	the public base point (generator)
a, b	Alice's and Bob's secret scalars
kP	scalar multiplication: P added to itself k times
ECDLP	elliptic-curve discrete logarithm problem (the hard one)

15 Side-channel attacks

§12 showed one way sound cryptography fails in practice: bad randomness. Side channels are the other. The cipher is mathematically unbroken — but its *execution* on a real machine leaks.

15.1 Attacking the implementation, not the algorithm

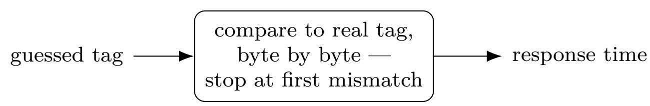
An attacker need not touch the front door — the inputs and outputs the math protects. Instead they watch a *side effect* of the computation: how long it took, what it did to the cache, how much power it drew, what it radiated. If any of those depends on secret data, the secret leaks.

These attacks are dangerous precisely because they sidestep the hard problems entirely. No factoring, no discrete log — just measurement.

15.2 Timing attacks

If an operation's *duration* depends on a secret, a stopwatch leaks the secret.

The canonical case is the one §3 warned about: comparing a received tag to the real one with an ordinary byte-by-byte compare that *returns at the first mismatch*. Then the response time grows with the number of matching leading bytes.



longer time ⇒ one more leading byte matched
⇒ keep that byte, then attack the next

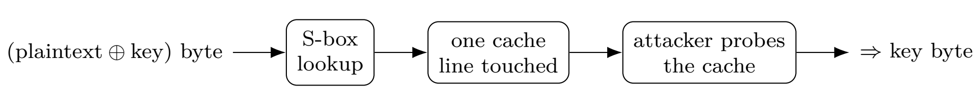
So an attacker forges a tag byte by byte: try all 256 values for byte 1, keep the one that runs slightly longer, move to byte 2, and so on — turning an infeasible 2^{128} guess into a few thousand tries.

The same trap hides in any secret-dependent branch (an `if` on a key bit), and in variable-time arithmetic — notably a square-and-multiply (§8) or double-and-add (§14) that skips steps on zero bits, leaking the private exponent or scalar. These work even remotely: RSA keys have been extracted over a network from timing alone.

15.3 Cache-timing attacks

A subtler timing channel runs through the CPU cache: data already cached is fast to read, data that must be fetched is slow. So if a computation reads a table at a *secret-dependent index*, whether that entry is cached leaks which index was used.

The classic target is AES's S-box (§6), implemented as a 256-byte lookup indexed by (plaintext ⊕ key) bytes:



An attacker sharing the machine primes the cache, lets AES run, then times their own reloads (the Flush+Reload technique) to see which table lines AES touched — recovering key bytes. This is exactly the weakness §5 and §6 named: it is why ChaCha20's table-free ARX is preferred in software, and why AES is safest on the AES-NI hardware instructions, which run a round with no data-dependent lookup.

15.4 Power and electromagnetic analysis

For a device the attacker can hold — a smart card, a hardware wallet, an embedded chip — the power it draws and the electromagnetic field it emits both track the operations and data inside.

Simple power analysis (SPA) reads the secret straight off one trace: a square-and-multiply leaves a visible pattern whose shape spells out the exponent bits.

Differential power analysis (DPA) is stronger — collect many traces and statistically correlate them against guesses of an intermediate value; the correct key guess stands out even when buried in noise.

(Related are *fault attacks*: deliberately glitch the chip — a voltage or clock spike, a laser — to force a wrong result that leaks the key. A single faulty RSA signature can reveal the factorisation.)

15.5 The defence: constant-time

One principle ties the software defences together: a crypto operation's running time and memory-access pattern must not depend on secret data — *constant-time* (more precisely, data-independent) code.

In practice that means: no secret-dependent branches (compute both sides and select with arithmetic); no secret-dependent memory addresses (no `table[secret]` — use ARX, bit-slicing, or hardware AES); and the constant-time comparison §3 required (XOR all the bytes together, test once at the end).

Design choices carry much of the load: ARX ciphers (ChaCha20, §5) have no tables to leak, X25519 (§14) was built for safe constant-time code, and AES-NI removes the S-box table. Power and EM, by contrast, need hardware countermeasures — masking (split each secret into random shares so no single measured point depends on it), plus shielding and added noise.

Channel	What it leaks	Defence
Timing	duration depends on the secret	no secret branches; constant-time compare
Cache	which table index was used	no secret-indexed tables (ARX, AES-NI)
Power / EM	the data and ops a chip runs	masking, shielding

The lesson is that security is a property of the whole system. "The cipher is unbroken" does not mean "your implementation is safe" — which is why constant-time has shadowed this entire document.

15.6 Symbols

No new persistent symbols. Local to this section:

Local term	Meaning (this section)
side channel	an observable effect (time, cache, power, EM) that depends on secrets
constant-time	code whose timing and memory access do not depend on secret data
SPA, DPA	simple / differential power analysis
Flush+Reload	a cache-probing technique behind cache-timing attacks

16 Key management

Every section so far opened with "a key" and assumed it was secret and present. But a key is an object with a *lifecycle* — it must be generated, stored, used, rotated, and destroyed — and in the real world most failures live here, not in the algorithms.

16.1 The forgotten layer

Keys committed to source control, left in logs, baked into firmware, never rotated, or lifted from an unprotected device — these break far more systems than any cipher weakness. The strongest algorithm in this document is worthless the moment its key leaks. This section is the operational layer beneath the math.

16.2 Generating keys

A symmetric key is just random bytes, drawn from a CSPRNG (§12) at the algorithm's length (e.g. 256 bits for AES-256). Never from a weak generator, and never use a password *directly* as a key.

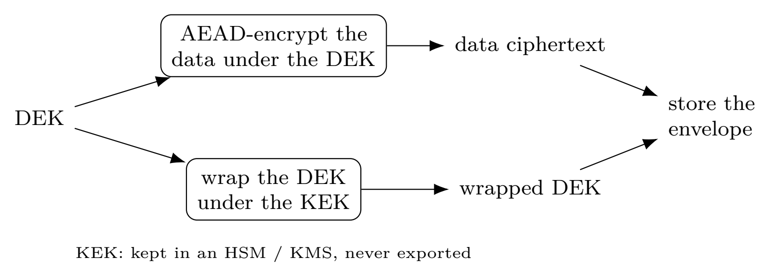
A password must first pass through a slow, salted password KDF (Argon2 or PBKDF2, from the hashing collection), which stretches a low-entropy secret into a key while resisting guessing.

Asymmetric keys are generated as a pair (§9, §13, §14): the private key from the CSPRNG, the public key derived from it.

16.3 Storage: the key hierarchy

Where should a key live so it is not simply sitting in a readable file? On a spectrum: in memory only (ephemeral session keys, §10 — safest, but gone on restart); encrypted on disk under another key; in a secrets manager; or in *hardware* — an HSM, TPM, or secure enclave, where the key never leaves. You feed data in and get results out, so even a compromised host (or a side channel, §15) cannot extract it.

But "encrypt the key under another key" only moves the problem — eventually some root key must rest in hardware or a human secret. The standard way to organise this is a key hierarchy with *envelope encryption*: a long-lived *key-encryption key* (KEK) wraps many short-lived *data-encryption keys* (DEKs); each DEK encrypts the actual data; and you store each data ciphertext alongside its wrapped DEK — the "envelope."



To read the data, unwrap the DEK with the KEK, then decrypt. Only the small KEK needs the strongest protection (an HSM or cloud KMS that never exports it); DEKs can be plentiful, per-file, generated freely, and rotated cheaply. This is exactly how cloud KMS services operate.

16.4 Rotation and destruction

Keys should be replaced periodically, and immediately on suspected compromise, for three reasons: to bound how much data rides on any one key (cryptographic wear — recall GCM's per-key message limit, §7), to limit the blast radius if a key leaks, and to meet policy.

Envelope encryption makes rotation cheap: to rotate the KEK, just re-wrap the DEKs under the new one — the bulk data, still under unchanged DEKs, is never touched. Tag each ciphertext with a key identifier so you know which key opens it.

To retire a key for good, destroy every copy. *Crypto-shredding* turns this into a feature: delete the key and the data it protected becomes permanently unreadable — a way to "erase" terabytes by erasing a few bytes.

For public keys, retire a compromised one by revoking its certificate (§9) — through revocation lists or OCSP — so it is no longer trusted.

16.5 Principles

A few rules underlie all of the above:

- **One key, one job.** Separate keys per purpose — encryption vs. MAC ($k_e \neq k_m$, §3), per-direction session keys (§10). A key's exposure must not spill across roles.
- **Minimise exposure.** Keys live in as few places as possible, ideally hardware; never written to logs, never committed to source control.
- **Plan for compromise.** Assume a key will eventually leak, and keep rotation, revocation, and crypto-shredding ready so the damage stays bounded.

16.6 Symbols

No new persistent symbols. Local to this section:

Local term	Meaning (this section)
KEK	key-encryption key: long-lived, wraps DEKs, kept in hardware
DEK	data-encryption key: per-data, encrypts the actual content
wrap / unwrap	encrypt / decrypt one key under another
HSM, KMS	hardware security module / key-management service
crypto-shredding	destroying data by destroying its key

17 The padding-oracle attack

§3 claimed that decrypting attacker-supplied ciphertext *before* verifying it is dangerous, and that MAC-then-Encrypt opened the door to "padding-oracle attacks." Here is that attack in full — decrypting CBC ciphertext byte by byte, with *no key*, from a single bit of feedback.

17.1 The setup: CBC, padding, and an oracle

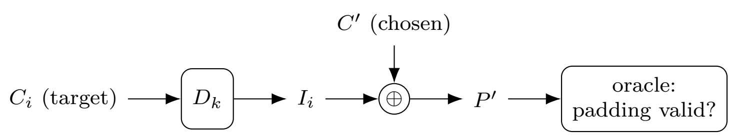
Recall CBC decryption (§2): $P_i = D_k(C_i) \oplus C_{i-1}$, with $C_0 = IV$. And PKCS#7 padding (§2.4): the last block ends with p bytes each equal to p, and on decryption the receiver checks that and rejects bad padding.

A *padding oracle* is anything that tells the attacker whether a submitted ciphertext decrypted to *valid padding* — a distinct error, a different response, or even a timing difference (a side channel, §15). That is one bit per query. It is enough.

17.2 The key idea: control the previous block

Write $I_i = D_k(C_i)$ for the *intermediate value* — the block cipher's decryption of C_i, *before* the XOR — so that $P_i = I_i \oplus C_{i-1}$. Crucially, I_i depends only on C_i and the key, so it is *fixed* for a given target block, even though the attacker cannot compute it.

The attacker submits a two-block ciphertext: a *chosen* block C' followed by the real target C_i. The decryptor produces, as the last "plaintext" block, $P' = I_i \oplus C'$ — and checks *its* padding.



Because the attacker controls C' byte by byte, they steer $P' = I_i \oplus C'$ (offset by the unknown I_i). So if they can learn I_i, the real plaintext follows for free: $P_i = I_i \oplus C_{i-1}$, and C_{i-1} is just known ciphertext.

17.3 Cracking the last byte

Vary the last byte of C' through all 256 values. For exactly one, the last byte of P' becomes `0x01` — valid one-byte padding — and the oracle says "valid." Then

$$P'[16] = I_i[16] \oplus C'[16] = \texttt{0x01} \implies I_i[16] = C'[16] \oplus \texttt{0x01}.$$

One byte of the intermediate value, recovered in at most 256 tries.

(Caveat: very rarely the "valid" answer is really a longer padding such as `0x02 0x02`, a false positive. Confirm by also flipping the second-to-last byte: genuine `0x01` padding is unaffected, a coincidental longer one breaks. A couple of extra queries settle it.)

17.4 Cracking the rest

Now climb inward. To get the next byte, aim for two-byte padding `0x02 0x02`: using the known $I_i[16]$, set $C'[16] = I_i[16] \oplus \texttt{0x02}$ so $P'[16] = \texttt{0x02}$, then brute-force $C'[15]$ until the oracle accepts. In general, to recover the k-th byte from the end, force the already-known trailing bytes to the padding value k and search the next byte.

step 1 (byte 16):	···	·	·	·	·	01
step 2 (byte 15):	···	·	·	·	02	02
step 3 (byte 14):	···	·	·	03	03	03

Orange is the byte being brute-forced to the shown value; gray is bytes already recovered, forced to it. Sixteen bytes fall in at most $256 \times 16 \approx 4096$ queries; then $P_i = I_i \oplus C_{i-1}$ gives the real block, and repeating per block decrypts the whole message — never once knowing the key.

17.5 Why it works, and the fix

The decryptor became an oracle: on attacker-supplied ciphertext it leaked a function of the secret intermediate value (was the padding valid?). That is exactly the failure §3 warned of — inspecting attacker data *after* decrypting but *before* (or instead of) verifying its integrity, the weakness of MAC-then-Encrypt.

The fix is authenticated encryption / Encrypt-then-MAC (§3): verify the tag over the ciphertext *first* and reject forgeries with $\perp$ before any decryption or padding check. A forged ciphertext never reaches the padding logic, so there is no oracle.

Merely hiding the signal — uniform error messages, constant-time padding checks (§15) — is only a band-aid: the Lucky 13 attack recovered the bit from timing alone. The real cure is AEAD.

This is not academic. Vaudenay introduced the attack in 2002; padding oracles went on to break SSL/TLS in CBC mode (POODLE, Lucky 13), ASP.NET, and more. It is the canonical reason modern systems use AEAD and never unauthenticated CBC.

17.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$C_i,\ C_{i-1}$	the target ciphertext block and the one before it (§2)
I_i	the intermediate value $D_k(C_i)$, before the CBC XOR
C'	the attacker's chosen "previous" block
P'	the last plaintext block the oracle checks, $P' = I_i \oplus C'$
padding oracle	a signal revealing whether decryption gave valid padding
k	the padding length being forged at each step

18 The 25519 curves: X25519 and Ed25519

§14 explained elliptic curves in general. In practice you do not invent a curve — you use a vetted, named one, and today that is overwhelmingly the *25519* family: X25519 for key agreement and Ed25519 for signatures.

18.1 Why a named curve

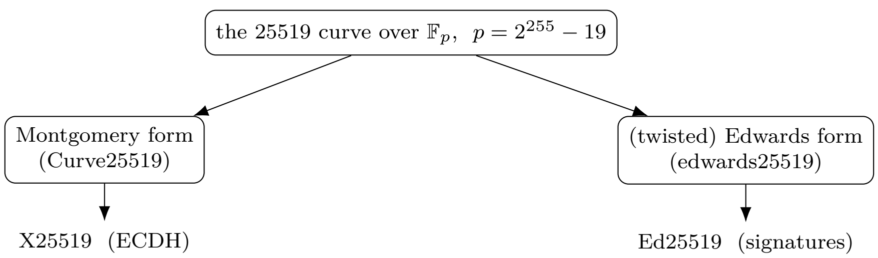
Rolling your own curve is as dangerous as rolling your own cipher. A bad choice can hide a weakness, and even a good curve is easy to implement insecurely. So everyone shares a few standard curves whose parameters and pitfalls are well studied.

The 25519 family (Bernstein) is the modern default for three reasons: it is fast, its parameters are *rigid* — derived transparently as the smallest values meeting the security criteria, a nothing-up-my-sleeve choice (§12) — and it is built to be hard to implement wrong. That contrasts with the older NIST P-curves, whose coefficients come from an unexplained seed, a provenance that bred distrust after Dual_EC (§12).

18.2 One curve, two forms

The name comes from the field: all arithmetic is mod the prime $p = 2^{255} - 19$ (a prime just below a power of two, which makes reduction fast), giving about 128-bit security with 32-byte keys.

The same underlying curve is written in two equivalent shapes, each suited to one job:



The *Montgomery* form (Curve25519) admits a fast, constant-time, x-coordinate-only scalar multiplication (the "Montgomery ladder"), ideal for ECDH. The *Edwards* form (edwards25519) has a *complete* addition law — one formula that works for every pair of points, with no special cases to leak timing — ideal for signatures. The two forms are the same curve under a change of coordinates.

18.3 X25519: key agreement

X25519 is a function, not a full point API. It takes a scalar and a u-coordinate (the Montgomery x) and returns a u-coordinate; public keys and shared secrets are just 32-byte u-values. It is ECDH (§14) with a deliberately small, safe interface.

With base point $u = 9$, Alice (scalar d_A) and Bob (scalar d_B) do:

$$\mathrm{pub}_A = \mathrm{X25519}(d_A, 9), \qquad \mathrm{shared} = \mathrm{X25519}(d_A, \mathrm{pub}_B) = \mathrm{X25519}(d_B, \mathrm{pub}_A).$$

Two safety features are built in. The scalar is *clamped* — a few bits forced — so it is a multiple of the cofactor and in a fixed range, sidestepping small-subgroup attacks. And every 32-byte string is an acceptable public key, so there are no invalid-curve checks to forget. This misuse-resistance is why X25519 is the default key exchange in TLS 1.3, SSH, Signal, and WireGuard.

18.4 ECDSA: the scheme Ed25519 improves on

To see why Ed25519 is built the way it is, look first at the older standard it replaces — ECDSA, the Elliptic Curve Digital Signature Algorithm. It uses the elliptic-curve key of §14: the private key is a secret integer (a *scalar*) d, and the public key is the *point* $Q = dG$ — the base point G added to itself d times (scalar multiplication, §14), easy to compute but infeasible to reverse. Here n is the order of G (§14) — the length of the cycle of multiples of G — so scalars are taken mod n. To sign a message M, hash it to a number $e = H(M)$ and pick a *fresh random* secret nonce k ($1 \le k < n$):

$$R = kG, \quad r = (x\text{-coordinate of } R) \bmod n, \quad s = k^{-1} \cdot (e + r\,d) \bmod n.$$

All of this is *modular* arithmetic — integers in $\{0, 1, \ldots, n-1\}$ taken mod n, never floating point. Juxtaposition like $r\,d$ is ordinary multiplication $r \cdot d$. And k^{-1} is the *modular inverse* of k *alone* — the unique integer with $k \cdot k^{-1} \equiv 1 \pmod{n}$ (found by the extended Euclidean algorithm, not a fraction) — which is *then multiplied* by the number $(e + r\,d)$. It is not the inverse of the whole product.

So computing s is two steps: first find k^{-1}, then form $k^{-1} \cdot (e + r\,d) \bmod n$. For instance with $n = 7$, $k = 3$, $e = 2$, $r = 4$, $d = 5$: the inverse is $k^{-1} = 5$ (since $3 \cdot 5 = 15 \equiv 1$), and $e + r\,d = 2 + 20 = 22$, so $s = 5 \cdot 22 = 110 \equiv 5 \pmod{7}$. Because n is prime, every nonzero value has an inverse, so s is always an integer in $\{0, \ldots, n-1\}$.

The signature is the pair (r, s). To verify with Q, compute $u_1 = e\,s^{-1}$ and $u_2 = r\,s^{-1} \pmod{n}$, and accept iff the x-coordinate of $u_1G + u_2Q$, reduced mod n, equals r — which works because $u_1G + u_2Q = s^{-1}(e + rd)\,G = kG = R$.

The whole danger lives in the nonce k: it must be unpredictable and used once. Suppose two signatures reuse the same k (so they share the same r):

$$s_1 = k^{-1}(e_1 + rd), \qquad s_2 = k^{-1}(e_2 + rd).$$

Subtracting clears d: $s_1 - s_2 = k^{-1}(e_1 - e_2)$, so $k = (e_1 - e_2)/(s_1 - s_2) \bmod n$ — the attacker reads the nonce straight off two public signatures. And once k is known, $d = (s_1k - e_1)/r \bmod n$ hands over the *private key*. Even a slightly biased or partly-leaked k recovers d by lattice methods. This is the footgun of §9 and §12, now in full: the nonce sits in an equation *with* the private key, so any weakness in k becomes a weakness in d.

18.5 EdDSA and Ed25519: deterministic signatures

EdDSA, the Edwards-curve Digital Signature Algorithm, is a different design — a *Schnorr* signature on a (twisted) Edwards curve — that closes ECDSA's two weak spots: signing needs no modular inverse, and the per-signature value is derived *deterministically* instead of chosen at random. Ed25519 is EdDSA on edwards25519 with SHA-512 as the hash H.

A private key is a 32-byte seed; hashing it splits into a clamped private scalar d and a secret *prefix*. The public key is the point $A = dB$, where B is the base point. To sign a message M:

$$r = H(\text{prefix} \parallel M), \quad R = rB, \quad h = H(R \parallel A \parallel M), \quad S = (r + h\,d) \bmod L,$$

with L the prime order of the base point. The signature is the pair (R, S) — 64 bytes. Verification accepts iff

$$S\,B = R + h\,A \qquad (\text{recomputing } h = H(R \parallel A \parallel M)),$$

since $SB = (r + hd)B = R + hA$. This shape — a commitment R, a challenge h, a response $S = r + hd$ — is exactly a Schnorr signature.

The decisive choice is that the per-signature value r is computed *deterministically* from the secret prefix and the message — there is *no random number generator at signing time*. That removes the ECDSA footgun at the root: with no random nonce to repeat or bias, the private-key-recovery disaster simply cannot occur.

	per-signature value	risk
ECDSA	a fresh random nonce k	reuse or bias $\rightarrow$ key recovery
Ed25519	deterministic $r = H(\text{prefix} \parallel M)$	none — no RNG involved

Together with small keys, fast constant-time arithmetic (the complete Edwards addition), and rigid parameters, this is why Ed25519 is the modern default signature in SSH, TLS 1.3, Signal, and software signing.

Notation flag. ECDSA above uses lowercase (r, s), random nonce k, public key Q, order n; Ed25519 uses uppercase (R, S), deterministic nonce r, challenge h, public key $A = dB$, base B, order L — all local, distinct from the A, B, d, s of earlier sections.

18.6 Symbols

No new persistent symbols. Local to this section:

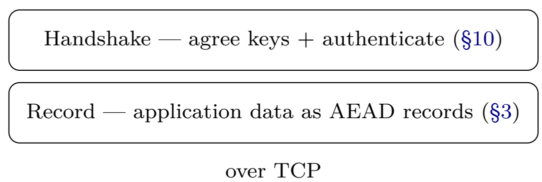
Local symbol	Meaning (this section)
p	the field prime $2^{255} - 19$
X25519	the ECDH function on Curve25519 (u-coordinates)
clamping	forcing a few scalar bits for safety (cofactor, range)
ECDSA	EC signature with a *random* nonce (§9)
$Q = dG,\ n$	ECDSA public key, and base-point order n
$e,\ k,\ (r, s)$	ECDSA message hash, random nonce, signature
Ed25519	EdDSA signature on edwards25519 with SHA-512
$B,\ A,\ d$	Ed25519 base point, public key $A = dB$, private scalar d
$r,\ R$	Ed25519 deterministic nonce $r = H(\text{prefix} \parallel M)$, $R = rB$
$h,\ (R, S)$	challenge $h = H(R \parallel A \parallel M)$; signature $S = (r + hd) \bmod L$
L	the prime order of the base point

19 TLS and the web PKI

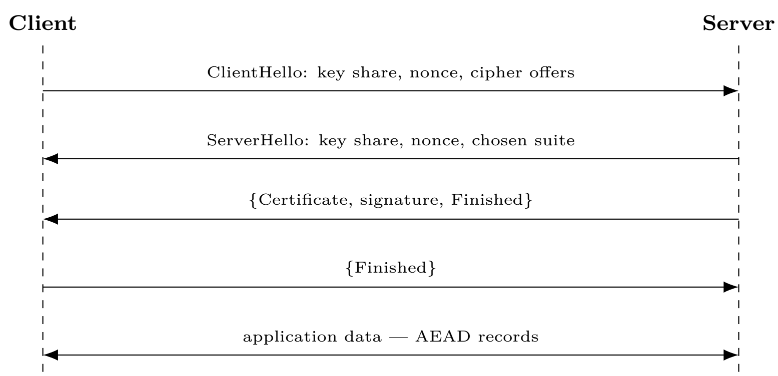
§10 built a secure channel (TLS its everyday example); §9 gave certificates. Here are the two real-world pieces: the TLS protocol *around* the handshake, and the *public-key infrastructure* (PKI) that decides whose certificates to trust.

19.1 TLS: the protocol around the handshake

Two layers — a *handshake* (agree keys, authenticate) and a *record* protocol (carry data as AEAD records):



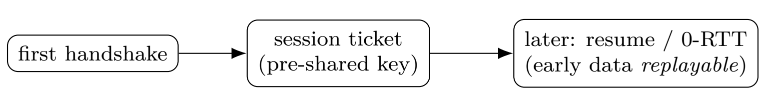
The TLS 1.3 handshake is one round trip; the client *offers* cipher suites and the server *picks* one ({ } marks encrypted messages):



The Finished MAC (§10) covers the whole transcript, so tampering with the negotiation — a *downgrade attack* — is caught.

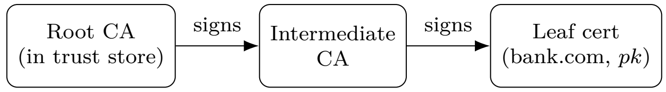
	TLS 1.2	TLS 1.3
Round trips	2	1
Key exchange	RSA or (EC)DHE	(EC)DHE only
Old/weak options	CBC, RC4, RSA	removed
Handshake	mostly in clear	mostly encrypted

A returning client can *resume* from a ticket, or send *0-RTT* data in the first message — fast, but that early data is *replayable*, so it must be safe to repeat:



19.2 The web PKI: who to trust

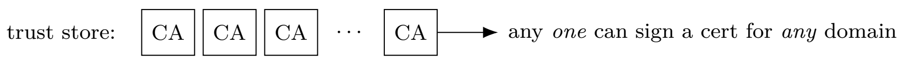
A signature proves *who* signed; PKI says *whose* signature to trust. Your OS or browser ships a *trust store* of a few hundred root CAs; roots sign intermediates, intermediates sign the leaf (server) certificate — so the server presents a chain:



The browser verifies each link up to a trusted root. Certificates use the *X.509* format (public key + domain + validity + issuer signature). A domain-validated certificate proves only *control of the domain* — you place a CA-named token in DNS or at a URL (automated by ACME / Let's Encrypt) — not who you are.

19.3 The weak link, and the fixes

Any of the hundreds of trusted CAs can issue a certificate for *any* domain, so one breached CA can impersonate any site (DigiNotar, 2011: forged Google certificates, then removed from every trust store):



The fixes:

Fix	What it does
Short-lived certs	expire before a leak matters
Certificate Transparency	public append-only logs $\Rightarrow$ mis-issuance is visible
CAA records	a DNS record limits which CAs may issue for a domain
Revocation (CRL / OCSP)	cancel a bad cert (in practice unreliable)

19.4 Symbols

No new persistent symbols. Local to this section:

Local term	Meaning (this section)
CA	certificate authority: an entity trusted to sign certificates
root / intermediate / leaf	the three levels of a certificate chain
trust store	the set of root CAs your OS or browser trusts by default
X.509	the certificate format binding a public key to an identity
CRL, OCSP	revocation: a published list, or an online status check
CT	Certificate Transparency: public append-only logs of issued certs
CAA	a DNS record naming which CAs may issue for a domain

20 NIST and the standards landscape

Most algorithms in this document come from one place: NIST, the US National Institute of Standards and Technology, whose cryptographic standards are used worldwide. It is worth knowing what it publishes, how, and why people both trust and occasionally distrust it.

20.1 What NIST publishes

Two document series:

Series	What	Examples (this document)
FIPS	the algorithms	AES (197), SHA-2 (180), SHA-3 (202), ECDSA (186), ML-KEM/-DSA, SLH-DSA (203/204/205)
SP 800	modes & guidance	GCM (800-38D), RNG (800-90A), key management

20.2 The open-competition model

NIST's flagship algorithms are chosen by public, multi-year *competitions*: the world submits candidates and tries to break them, and the survivor becomes the standard. That open cryptanalysis is what earns the trust.

Competition	Years	Winner	Where
AES	1997–2001	Rijndael	§6
SHA-3	2007–2012	Keccak	hashing collection
Post-quantum	2016–2024	Kyber, Dilithium, SPHINCS$^+$	§11

20.3 The trust question

NIST is a US government body, and that has twice bred suspicion:

- **Dual_EC_DRBG** (SP 800-90A, ∼2006): a standardised generator whose constants could hide a backdoor (§12); eventually withdrawn.
- the **NIST P-curves** (P-256, …): parameters from an unexplained seed — so, after Dual_EC, many prefer the transparently chosen Curve25519/Ed25519 (§18).

The lesson: an open competition earns trust; opaque constants lose it — hence the value of nothing-up-my-sleeve parameters (§6, §12).

20.4 NIST, or not?

You usually pick a NIST algorithm — and *must*, for US-government or FIPS-140 compliance. The popular alternatives are IETF or academic:

Job	NIST	Common alternative
AEAD	AES-GCM	ChaCha20-Poly1305
Key exchange	ECDH P-256	X25519
Signatures	ECDSA P-256	Ed25519
Hashing	SHA-2 / SHA-3	BLAKE3

20.5 Symbols

No new persistent symbols. Local terms:

Local term	Meaning (this section)
FIPS	Federal Information Processing Standard — a NIST-published algorithm
SP 800	NIST Special Publication — modes, RNG, and guidance
FIPS 140	the validation program a crypto module passes to be “FIPS-approved”

21 Modular arithmetic

Public-key cryptography runs on *modular arithmetic* — integers wrapped around a modulus. We have leaned on it in Diffie–Hellman (mod p), ECDSA (mod n), and the fields of §4. Here it is in one place.

21.1 Wrapping at n

$a \bmod n$ is the remainder when a is divided by n — arithmetic that wraps around like a clock (on a 12-hour clock, $10 + 5 = 3$). Write $a \equiv b \pmod{n}$ when n divides $a - b$. The values live in $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$.

21.2 The operations

Add, subtract, and multiply by doing the operation and taking the remainder; the result stays in $\mathbb{Z}_n$. Powers use square-and-multiply (§8); division is multiplication by an inverse (below).

Operation	Rule	Example (mod 7)
Add	$(a + b) \bmod n$	$5 + 4 = 9 \equiv 2$
Multiply	$(a \cdot b) \bmod n$	$5 \cdot 4 = 20 \equiv 6$
Power	$g^k \bmod n$ (square-and-multiply)	$3^4 = 81 \equiv 4$
Inverse	a^{-1}: $a \cdot a^{-1} \equiv 1$	$3^{-1} = 5$ $(3 \cdot 5 = 15 \equiv 1)$

21.3 Inverses and fields

The *modular inverse* a^{-1} is the element with $a \cdot a^{-1} \equiv 1 \pmod{n}$ — it is how you "divide" (no fractions, no floats). It exists exactly when $\gcd(a, n) = 1$, found by the extended Euclidean algorithm (or, when n is prime, as a^{n-2}, by Fermat).

When the modulus is a *prime* p, *every* nonzero element has an inverse, so $\mathbb{Z}_p$ is a field — the $\mathrm{GF}(p)$ of §4. Composite moduli are not fields (e.g. 2 has no inverse mod 4).

21.4 Why crypto uses it

Two reasons: it is *finite and exact* — every value an integer in $\{0, \ldots, n-1\}$, no rounding or floating point — and it hides *one-way* problems. $g^k \bmod p$ is cheap to compute, but recovering k (the discrete logarithm) is infeasible, which is what Diffie–Hellman (§8) and ECDSA (§9) stand on.

21.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$a \bmod n$	the remainder of a divided by n
$a \equiv b \pmod{n}$	n divides $a - b$ (same remainder)
$\mathbb{Z}_n$	the integers mod n: $\{0, 1, \ldots, n-1\}$
a^{-1}	modular inverse: $a \cdot a^{-1} \equiv 1 \pmod{n}$
$\gcd(a, n)$	greatest common divisor (inverse exists iff $= 1$)

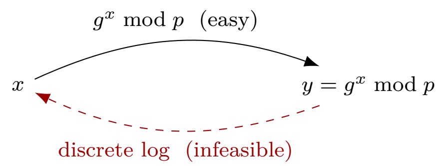
22 The discrete logarithm problem

The hard problem behind Diffie–Hellman (§8) and (EC)DSA (§9): easy one way, infeasible the other.

Fix a public prime p and base g. Computing

$$y = g^x \bmod p$$

from x is cheap (square-and-multiply, §21). The *discrete logarithm problem* (DLP) is the reverse — given g, p, y, recover $x = \log_g y$ — and no efficient method is known.



Toy: mod 23 with $g = 5$, from $x = 6$ we get $y = 5^6 \bmod 23 = 8$ at once; going back from $y = 8$ to $x = 6$ means searching — trivial at 23, hopeless at 3072 bits.

Setting	Best attack	Size for ∼128-bit security
mod a prime p	index calculus (sub-exponential)	∼3072-bit p
elliptic curve (ECDLP)	Pollard rho (square-root)	∼256-bit

The elliptic-curve version (ECDLP, §14, §18) is harder per bit, which is why curve keys are so much smaller. A quantum computer breaks both: Shor's algorithm solves the DLP efficiently (§24, §11), so DH and (EC)DSA need post-quantum replacements.

22.1 Symbols

No new persistent symbols. Local to this section:

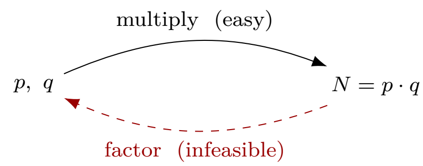
Local symbol	Meaning (this section)
$g,\ p$	public base and prime modulus
x	the secret exponent (the discrete log)
$y = g^x \bmod p$	the public value
$\log_g y$	the discrete logarithm: the x with $g^x \equiv y$
ECDLP	the discrete log problem on an elliptic curve

23 The integer factorization problem

The hard problem behind RSA (§9, §13): multiplying two large primes is cheap; splitting the product back apart is not.

$$N = p \cdot q \qquad (p, q \text{ large primes}).$$

Given p, q, computing N is instant. *Integer factorization* is the reverse — given N, recover p and q — and it is infeasible for large N.



Toy: $7 \cdot 11 = 77$ in a moment; factoring 77 back is also easy — but a 2048-bit N is not.

RSA's trapdoor. The public key is (N, e); the private key is d with $e\,d \equiv 1 \pmod{\varphi(N)}$, where $\varphi(N) = (p-1)(q-1)$ requires the factorization. So whoever can factor N recovers d — breaking RSA is, in essence, factoring N.

Best attack	Size for ∼128-bit security
general number field sieve (sub-exponential)	∼3072-bit N

(RSA-2048 gives about 112-bit security and is common today.) As with the discrete log, a quantum computer wins: Shor's algorithm factors efficiently (§24, §11), so RSA falls.

23.1 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$N = p \cdot q$	the public modulus, a product of two large primes
$p,\ q$	the secret prime factors
$e,\ d$	RSA public and private exponents, $e\,d \equiv 1 \pmod{\varphi(N)}$
$\varphi(N)$	$(p-1)(q-1)$ — computable only with the factorization

24 Shor's algorithm

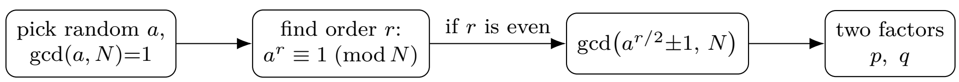
§22 and §23 each ended on the same dashed red arrow: a quantum computer makes the "infeasible" direction feasible. This is that machine. One idea sits under both attacks — *period-finding* — and both factoring and discrete log reduce to it.

24.1 Step 1: factoring becomes order-finding

To factor $N = p \cdot q$, pick a random a with $\gcd(a, N) = 1$ and find the *order* of a — the smallest $r > 0$ with

$$a^r \equiv 1 \pmod{N} \qquad \text{(the period of } f(x) = a^x \bmod N\text{)}.$$

This r is the only hard part. Once you have it, two greatest-common-divisors finish the job:



Why the gcd trick works. From $a^r \equiv 1$ we get $a^r - 1 \equiv 0$, and factoring the left side,

$$\big(a^{r/2} - 1\big)\big(a^{r/2} + 1\big) \equiv 0 \pmod{N}.$$

So N divides the product. If N divides *neither* factor on its own (that is the condition r even and $a^{r/2} \not\equiv -1$), then N must split its prime factors between the two — and $\gcd(a^{r/2} \pm 1, N)$ peels off a real factor.

Toy: $N = 15$, $a = 7$. The powers mod 15 are $7,\ 4,\ 13,\ 1, \ldots$, so $r = 4$ (even). Then $a^{r/2} = 7^2 = 49 \equiv 4$, and

$$\gcd(4-1,\ 15) = \gcd(3, 15) = 3, \qquad \gcd(4+1,\ 15) = \gcd(5, 15) = 5.$$

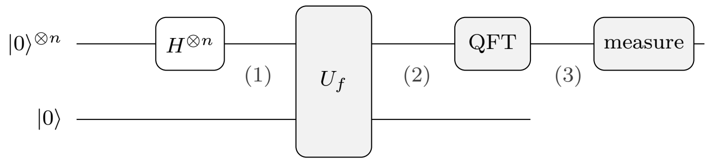
Factors 3 and 5 — recovered, given only the period r.

24.2 Step 2: the quantum period-finder

Finding r classically means essentially searching, which is why §23 called it infeasible. A quantum computer finds it by interference. Three terms, glossed once:

- a *qubit* is a bit that can hold 0 and 1 at once, in a weighted combination;
- the bracket $|\cdot\rangle$ (a *ket*) just wraps a quantum state: $|0\rangle$ is a qubit holding 0, and $|0\rangle^{\otimes n}$ is a row of n such qubits — the superscript $\otimes n$ ($\otimes$ is the *tensor product*) reads "n copies side by side". A gate written $H^{\otimes n}$ is likewise that one gate applied to each of the n qubits;
- a *superposition* $\sum_x \alpha_x |x\rangle$ is a state holding many integers x simultaneously, each with an amplitude α_x;
- a *register* is a row of qubits, here holding an integer.

The circuit uses two registers: a *counting* register of n qubits, and a *work* register that will hold $a^x \bmod N$. Just as n classical bits take 2^n different values, the counting register ranges over $Q = 2^n$ integers $0, 1, \ldots, Q-1$ — so Q is one ordinary whole number (a *count*, not a vector), and 2^n is plain exponentiation, e.g. $n = 3$ gives $Q = 2^3 = 8$ with $x \in \{0, \ldots, 7\}$:



Reading left to right, each gate is one stage. We write the system's state as $|\psi\rangle$, the usual symbol for a quantum state, subscripted by stage — so $|\psi_1\rangle$ is the state at point (1), $|\psi_2\rangle$ at (2), and so on:

(1) Hadamards. The *Hadamard gate* H is the basic superposition-maker: it turns a definite $|0\rangle$ into the even mix $\frac{1}{\sqrt{2}}\big(|0\rangle+|1\rangle\big)$. Applying one to each of the n counting qubits ($H^{\otimes n}$) spreads $|0\rangle^{\otimes n}$ into an equal superposition of *all* Q integers the register can hold, each with amplitude $1/\sqrt{Q}$ (so the Q equal terms' probabilities sum to 1):

$$|\psi_1\rangle = \frac{1}{\sqrt{Q}}\sum_{x=0}^{Q-1}|x\rangle\,|0\rangle.$$

(2) U_f computes $f(x) = a^x \bmod N$ into the work register — all Q values *at once*, the point of the superposition:

$$|\psi_2\rangle = \frac{1}{\sqrt{Q}}\sum_{x=0}^{Q-1}|x\rangle\,|a^x \bmod N\rangle.$$

Because f has period r (we have $a^{x+r} \equiv a^x$), the values of x sharing one work value a^{x_0} are exactly $x_0,\ x_0+r,\ x_0+2r,\ldots$ So the counting register now holds an evenly spaced *comb* of spikes, r apart:

$$|\psi_2'\rangle \ \propto\ \sum_j |x_0 + j\,r\rangle.$$

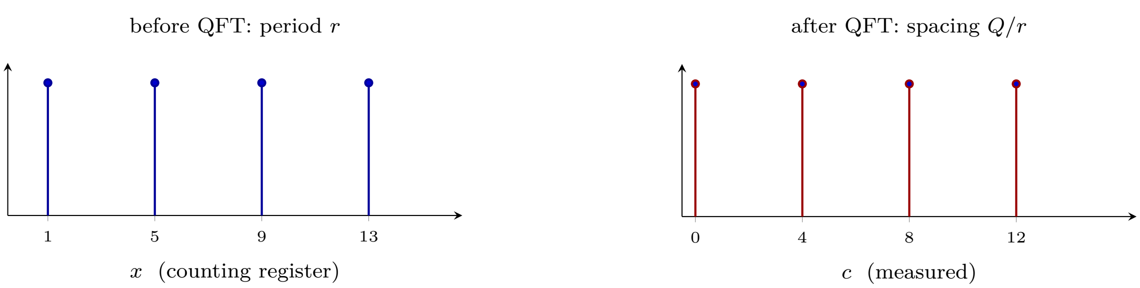
(3) QFT is the quantum discrete Fourier transform,

$$\mathrm{QFT}:\ |x\rangle \ \mapsto\ \frac{1}{\sqrt{Q}}\sum_{c=0}^{Q-1} e^{2\pi i\,xc/Q}\,|c\rangle,$$

and like any Fourier transform it turns a *period* into sharp *frequency lines*. The comb spaced r apart becomes a comb spaced Q/r apart. The one-line reason is a geometric sum: plugging the progression $x = x_0 + jr$ into the transform,

$$\text{amplitude at } c \ \propto\ \sum_j e^{2\pi i\,(x_0+jr)c/Q} = e^{2\pi i\,x_0c/Q}\sum_j \big(e^{2\pi i\,rc/Q}\big)^j,$$

and that sum of unit vectors is large only when rc/Q is a whole number — i.e. when c is a multiple of Q/r; otherwise the vectors point every which way and cancel.



(Here $Q = 16$, $r = 4$, so $Q/r = 4$ divides evenly and the peaks are exact. For a general r that does not divide Q the peaks are slightly smeared, near multiples of Q/r.)

24.3 Step 3: read off the period

Measuring the counting register gives one value c near a multiple of Q/r:

$$\frac{c}{Q} \approx \frac{k}{r} \qquad (k \text{ some unknown integer}).$$

You know c and Q but not k or r. Reduce the fraction c/Q to lowest terms with the *continued-fraction* expansion; its denominator is the candidate r. (Choosing $Q \geq N^2$ makes this denominator unique.) Check $a^r \equiv 1 \pmod{N}$; if it fails — or r is odd, or $a^{r/2} \equiv -1$ — start over with a fresh random a. A handful of tries succeeds with high probability.

24.4 Discrete log falls the same way

The discrete log of §22 is also a hidden period. To solve $g^x \equiv y$ in a group of order n, Shor finds the period of the two-input function

$$f(b, c) = g^b\, y^{-c},$$

whose periodicity pins down x (recovered by a two-dimensional QFT). This needs only the group operation, so it works in *any* group — including the elliptic-curve group (§14, §18). The curve's smaller keys buy nothing here. So Diffie–Hellman (§8) and every signature scheme of §9 break together.

24.5 Why it is fast — and why your keys are still safe today

First, what "fast" is measured against. The number N is enormous, but you write it with just $n = \log_2 N$ bits, and that bit-count *is* the key length (RSA-2048 means $n = 2048$). So the fair yardstick is how the work grows with n — the size you actually choose — not with the astronomical N. Three growth rates matter, and the sharpest way to feel them is to ask what *one extra key bit* costs the attacker:

Growth	Cost in n bits	One more bit	Seen in
polynomial	n^c (e.g. n^2, n^3)	barely changes	Shor
sub-exponential	between (exponent $\sim n^{1/3}$)	rises, but gently	NFS, index calculus
exponential	$2^{c\,n}$ (e.g. 2^n)	roughly *doubles*	brute force

So "polynomial in $\log N$" just means "polynomial in the number of bits n", since $n = \log_2 N$ — the cheap top row. Doubling the key length only multiplies Shor's work by a fixed factor (about $\times 4$ for n^2).

Exponential is the wall we *want* an attacker to hit: every added bit roughly doubles the work, so brute force never catches up. Sub-exponential sits between — worse than any polynomial, but milder than exponential, its exponent climbing like a fractional power of n (think $n^{1/3}$) rather than like n.

Shor's cost is the polynomial row (the modular exponentiation in U_f dominates). On the very scale where classical factoring and discrete log are only sub-exponential — slow enough that RSA-2048 and its peers still stand today — Shor is polynomial, which is what collapses them:

Problem	Best classical	Shor (quantum)
factor N (§23)	number field sieve, sub-exponential	polynomial in $\log N$
discrete log mod p (§22)	index calculus, sub-exponential	polynomial in $\log p$
ECDLP, 256-bit (§14)	Pollard rho, $\sim 2^{128}$	polynomial

The catch is the hardware. Shor needs a *fault-tolerant* quantum computer: millions of noisy physical qubits welded by quantum error correction into a few thousand stable logical qubits, running a long coherent circuit. Nothing close exists. That is exactly why the threat is "harvest now, decrypt later" (§11) — and why the migration to post-quantum schemes (§11) has to begin before the machine does.

24.6 Symbols

No new persistent symbols. Local to this section:

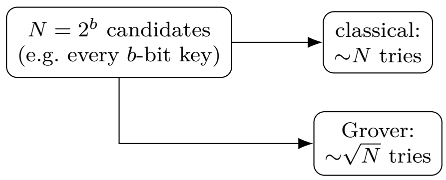
Local symbol	Meaning (this section)
$N = p \cdot q$	the number to factor
a	random base, $\gcd(a, N) = 1$
r	the order (period): smallest $r > 0$ with $a^r \equiv 1 \pmod N$
$Q = 2^n$	size of the counting register ($Q \geq N^2$, so r resolves)
$\|x\rangle$	quantum basis state for the integer x; $\sum_x \|x\rangle$ holds all x at once
U_f	the gate computing $f(x) = a^x \bmod N$ into the work register
QFT	quantum Fourier transform: period $r \mapsto$ spikes at multiples of Q/r
c	the measured value, $\approx k\, Q/r$

25 Grover's algorithm

Shor (§24) won by exploiting *structure* — the periodicity hiding inside factoring and discrete log. A secret key has no such structure: finding it is blind search through 2^b possibilities. Grover's algorithm is the best a quantum computer can do against that, and "best" here is only a *square-root* speedup — which is why the symmetric world (§11) survives by merely doubling key sizes.

25.1 The problem: unstructured search

A predicate f marks the item we want among N candidates — $f(x) = 1$ for the answer, 0 for everything else — and f tells us nothing about *where* the answer is. Classically you can only try candidates one by one: up to N checks, $N/2$ on average.



The crypto instance: brute-forcing a b-bit key is searching $N = 2^b$ candidates, with $f(k) = 1$ when key k decrypts the captured ciphertext to something sensible. Grover cuts the 2^b tries to about $2^{b/2}$.

25.2 How it works: amplitude amplification

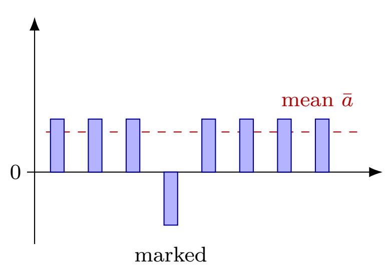
Start in the uniform superposition — every candidate present with the same amplitude $1/\sqrt{N}$:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

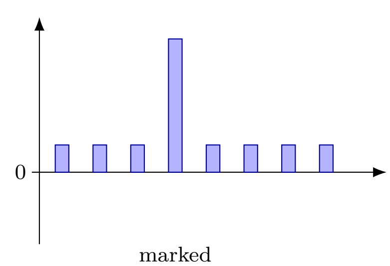
Then repeat one two-step move, the *Grover iteration*, about $\frac{\pi}{4}\sqrt{N}$ times:

- **Oracle** O — flip the *sign* of the marked amplitude, leaving the rest alone: $|x\rangle \mapsto (-1)^{f(x)}|x\rangle$.
- **Diffusion** D — *invert every amplitude about their mean* $\bar{a}$: $a_x \mapsto 2\bar{a} - a_x$. As an operator, $D = 2|s\rangle\langle s| - I$.

The sign flip drops the marked amplitude below the mean; reflecting about the mean then launches it far above the others. One iteration, in two frames ($N = 8$, the item at index 3 marked):



after oracle: marked sign flipped



after diffusion: marked amplified

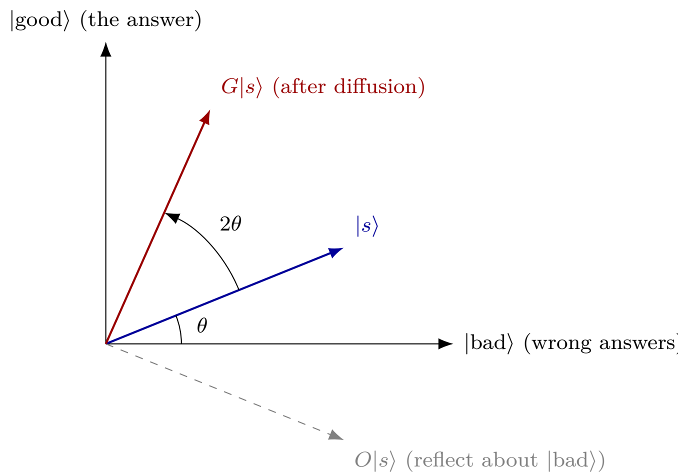
Each iteration pumps a little more amplitude into the marked state; after about $\frac{\pi}{4}\sqrt{N}$ of them, measuring almost always returns the answer.

25.3 The geometry: it is just a rotation

Why exactly $\frac{\pi}{4}\sqrt{N}$? Because the whole process is a steady rotation in a flat 2-D plane. Split every state into two parts: $|\text{good}\rangle$ (the marked answer) and $|\text{bad}\rangle$ (the even mix of all $N-1$ wrong answers). The starting state $|s\rangle$ sits at a tiny angle θ off $|\text{bad}\rangle$, with

$$\sin\theta = \frac{1}{\sqrt{N}} \;\approx\; \theta \quad (\text{for large } N).$$

The oracle is a reflection about the $|\text{bad}\rangle$ axis; diffusion is a reflection about $|s\rangle$. Two reflections make a *rotation*, by 2θ toward $|\text{good}\rangle$:



After t iterations the state stands at angle $(2t+1)\theta$. To land on $|\text{good}\rangle$ we want that angle near $\frac{\pi}{2}$, so

$$(2t+1)\,\theta \approx \frac{\pi}{2} \quad\Longrightarrow\quad t \approx \frac{\pi}{4\theta} \approx \frac{\pi}{4}\sqrt{N}.$$

Note you can *overshoot*: keep going and the state rotates *past* $|\text{good}\rangle$ and the success probability falls again — so the iteration count is tuned, not "more is better."

25.4 What it means for crypto

Searching $N = 2^b$ in $\sqrt{N} = 2^{b/2}$ steps halves the exponent — the effective security in bits:

Target	Classical	Grover	Fix
AES-128 key (§6)	2^{128}	2^{64}	use AES-256
AES-256 key	2^{256}	2^{128}	already fine
ChaCha20 key (§5)	2^{256}	2^{128}	already fine
hash preimage, n-bit	2^n	$2^{n/2}$	use a longer hash

The repair is trivial: double the key or output size and the quantum attacker is back to square one. No new mathematics — unlike the public-key half, which Shor demolishes outright and which must be *replaced* (§11).

Why only a square root — and why that is the ceiling. Grover is provably *optimal*: no quantum algorithm can search a structureless space of N in fewer than about $\sqrt{N}$ queries. With no pattern to exploit, $\sqrt{N}$ is all the quantum world buys — the exact opposite of Shor's exponential win on the *structured* problems of §24. (In practice the threat is even softer: Grover parallelises poorly

— splitting it across m machines saves only a factor $\sqrt{m}$ — and needs one very long coherent computation, so the "halving" is an upper bound on the danger, not a tight one.)

25.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$N = 2^b$	size of the search space (b-bit key $\Rightarrow 2^b$ candidates)
f	oracle predicate: $f(x) = 1$ iff x is the sought item
$\lvert s\rangle$	uniform superposition over all N candidates, amplitude $1/\sqrt{N}$
$O,\ D$	oracle (sign-flip) and diffusion (invert-about-the-mean) operators
$\bar{a}$	the mean amplitude (diffusion reflects each a_x about it)
θ	half the per-iteration rotation; $\sin\theta = 1/\sqrt{N}$
$t \approx \frac{\pi}{4}\sqrt{N}$	number of iterations to reach the answer

26 Quantum key distribution

Shor (§24) and Grover (§25) turned quantum mechanics into a weapon. Quantum key distribution (QKD) turns it into a shield: a way for two parties to agree on a secret key whose secrecy is guaranteed by the *laws of physics*, with any eavesdropping left as a detectable scar.

Two warnings before the mechanics, because QKD is widely misunderstood:

- QKD is **not** post-quantum cryptography. Post-quantum schemes (§11) are classical math run in software; QKD is physical hardware sending single photons down a fibre. They solve the same problem by opposite means.
- QKD distributes a **key only**. You still encrypt with a classical symmetric cipher (§6) afterwards, and — crucially — QKD needs a separately *authenticated* channel to work at all.

26.1 The idea: secrecy from physics

Classical key agreement like Diffie–Hellman (§8) is secure only because a computation is *hard*, and tapping the wire is undetectable. QKD leans on two physical facts instead:

- **No-cloning:** an unknown quantum state cannot be copied. Eve cannot duplicate a photon and measure the copy at leisure.
- **Measurement disturbs:** reading a photon in the wrong basis randomises the outcome *and* destroys the original. Eve cannot look without leaving traces.

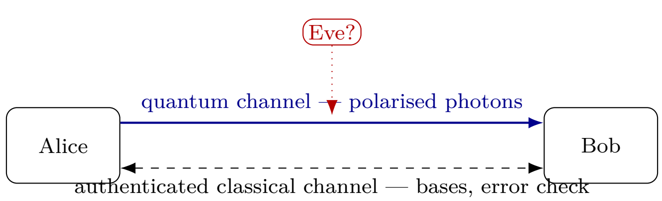
A bit is encoded in a photon's polarisation, using *two* bases — and that choice of two is the whole trick:

basis + (rectilinear) basis × (diagonal)

A photon prepared in one basis, then measured in the *other*, gives a purely random result — the original bit is gone. Reading correctly requires knowing which basis was used.

26.2 BB84, step by step

The original protocol (Bennett & Brassard, 1984) runs over two channels: a quantum channel carrying photons, and an authenticated classical channel for the public bookkeeping.



1. **Send.** For each photon Alice picks a random bit *and* a random basis, and sends the matching polarisation.

2. **Measure.** Bob picks a random basis for each photon and measures. Where his basis matches Alice's he reads her bit; where it does not, he gets noise.
3. **Sift.** Over the classical channel they announce only their *bases* (never the bits) and keep only the positions where the bases agreed — the *sifted key*, about half the photons.
4. **Check.** They publicly compare a random sample of sifted bits to estimate the error rate. Low $\Rightarrow$ no eavesdropper, discard the revealed bits and keep the rest. High $\Rightarrow$ abort.

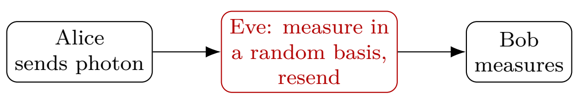
A short run (photon arrows: $\rightarrow, \uparrow$ in basis $+$; $\nearrow, \nwarrow$ in basis $\times$):

Alice bit	Alice basis	photon	Bob basis	Bob bit	bases match?	sifted key
0	$+$	$\rightarrow$	$+$	0	yes	0
1	$\times$	$\nwarrow$	$+$	1	no	—
1	$+$	$\uparrow$	$+$	1	yes	1
0	$\times$	$\nearrow$	$\times$	0	yes	0
1	$\times$	$\nwarrow$	$+$	0	no	—
0	$+$	$\rightarrow$	$\times$	1	no	—
1	$+$	$\uparrow$	$+$	1	yes	1
0	$\times$	$\nearrow$	$\times$	0	yes	0

The kept bits $0, 1, 0, 1, 0$ are shared secret — Alice and Bob hold them, but they were never spoken aloud, only the bases were.

26.3 Why Eve gets caught

No-cloning forces Eve to *measure* each photon to learn anything — but she does not know the basis, so she guesses. The classic intercept-resend attack:



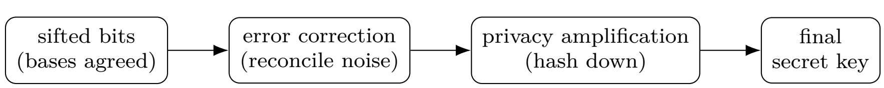
When Eve guesses the wrong basis she collapses the photon, so even when Bob's basis *does* match Alice's, his bit is now wrong half the time. On the sifted key this injects errors at a rate

$$\mathrm{QBER}_{\text{intercept-resend}} = \underbrace{\tfrac{1}{2}}_{\text{Eve's basis wrong}} \times \underbrace{\tfrac{1}{2}}_{\text{then Bob errs}} = \tfrac{1}{4} = 25\%.$$

A clean channel shows only a few percent of error from device noise, so a jump toward 25% is unmistakable: Alice and Bob see it in step 4 and abort before any key is used. Eve cannot do better, because copying the photon is physically forbidden.

26.4 From sifted bits to a usable key

Even with no Eve, the sifted bits carry some device noise and may leak a sliver of information. Two classical post-processing steps finish the key:



Privacy amplification shrinks the reconciled string with a hash, shrinking any partial knowledge Eve might hold to effectively nothing. What survives feeds a symmetric cipher (§6) exactly like any other key.

26.5 QKD versus post-quantum cryptography

Both defend key agreement against a quantum adversary, by opposite philosophies:

	QKD	Post-quantum crypto (§11)
Security from	laws of physics (no-cloning)	math believed hard for quantum
Medium	photons over fibre / free space	software on existing networks
Range	∼hundreds of km; trusted nodes	global, any IP path
Solves	key agreement only	key agreement *and* signatures
Authentication	still needs a classical one	provides its own (signatures)
Eavesdropping	detectable	not detectable (but infeasible)
Cost	special hardware, niche	drop-in, scalable

The decisive catch is authentication. The classical channel must be authenticated, or Eve sits in the middle running one QKD session with each side and relays between them. That authentication needs a pre-shared secret or classical/post-quantum signatures — so QKD does not create trust from nothing; it *expands* an existing shared secret.

For that reason, plus the cost, range limits, and side-channel attacks on real photon hardware, agencies such as the NSA and the UK NCSC do not recommend QKD for general use and point to post-quantum cryptography (§11) instead. QKD is real and deployed in niches, but it complements classical cryptography rather than replacing it.

26.6 Symbols

No new persistent symbols. Local to this section:

Local symbol / term	Meaning (this section)
BB84	the Bennett–Brassard 1984 QKD protocol
basis +	rectilinear: $\rightarrow$ encodes 0, $\uparrow$ encodes 1
basis ×	diagonal: $\nearrow$ encodes 0, $\nwarrow$ encodes 1
no-cloning	physical law: an unknown quantum state cannot be copied
sifted key	bits kept where Alice's and Bob's bases agreed
QBER	quantum bit error rate — the fraction of mismatched sifted bits

27 Lattice-based cryptography

The post-quantum replacements of §11 — ML-KEM, ML-DSA — rest on *lattices*. §11 named the idea; this section opens it up: what a lattice is, why its hard problems survive a quantum computer (unlike the targets of §24), and how *Learning With Errors* turns geometry into working encryption.

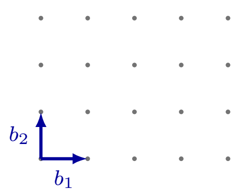
27.1 What is a lattice?

Take a few vectors $b_1, \dots, b_n$ (a *basis*) and form every *integer* combination of them:

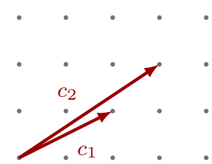
$$\mathcal{L} = \Big\{ a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \ : \ a_i \in \mathbb{Z} \Big\}.$$

The result is a lattice — an infinite, perfectly regular grid of points. (Integer coefficients are the whole point: real coefficients would fill the entire plane, not a discrete grid.)

The catch that makes lattices useful for cryptography: *the same lattice has many bases.* Some are *good* — short, nearly perpendicular vectors — and some are *bad* — long, skewed vectors. Both describe the identical set of points.



good basis: short, near-orthogonal

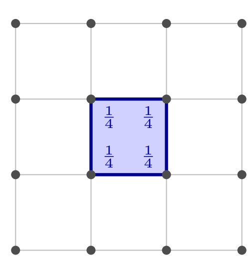


bad basis: long, skewed

Both bases generate the *same* lattice $\mathbb{Z}^2$ (the dots). Spelled out, the good basis is the pair of *column* vectors $b_1 = (1,0)^\top$, $b_2 = (0,1)^\top$ and the bad basis is $c_1 = (2,1)^\top$, $c_2 = (3,2)^\top$ — the $(\cdot)^\top$ marks a column, where a plain (x_1, x_2) would be a row. The test for "same lattice" is the *determinant* of the basis: place its two vectors as the columns of a matrix and take the 2×2 determinant,

$$\det[\, b_1 \quad b_2 \,] = \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} = 1 \cdot 1 - 0 \cdot 0 = 1, \qquad \det[\, c_1 \quad c_2 \,] = \begin{vmatrix} 2 & 3 \\ 1 & 2 \end{vmatrix} = 2 \cdot 2 - 3 \cdot 1 = 1.$$

That determinant is the *area* of the cell (the parallelogram) the basis spans. Now tile the whole plane with copies of that cell and count the lattice points: there is exactly *one per cell.* It looks like four — the corners — but every corner is shared by the four cells meeting there, so a single cell owns only a *quarter* of each corner, and $4 \times \frac{1}{4} = 1$:



one cell $= 4 \times \frac{1}{4} = 1$ point

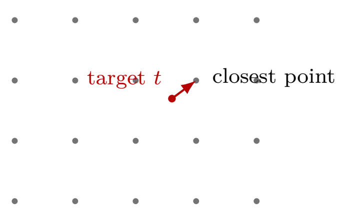
So the basis's points sit at a density of *one per cell-area.* The plain grid $\mathbb{Z}^2$ has one point per unit area, so an area-1 basis matches that density exactly and must land on *every* integer point — it is all of $\mathbb{Z}^2$. A basis of area 2 is half as dense (one point per two units of area), so it can reach only every *other* point: a sparser, *different* lattice. That is why $|\det| = 1$ is precisely the condition for "the same lattice".

In lattice cryptography the **good basis is the private key** and a **bad basis is the public key** — everyone can name the points, only the key-holder has the convenient handle on them.

27.2 The hard problems: SVP and CVP

Two questions about a lattice are easy with a good basis and brutally hard with a bad one in high dimension:

- **SVP** (shortest vector problem): find the shortest nonzero vector in $\mathcal{L}$.
- **CVP** (closest vector problem): given a target point t *not* on the lattice, find the nearest lattice point.



With a *good* basis, CVP is just rounding: write t in that basis and round each coordinate ($2.6 \to 3$, $1.7 \to 2$). With a *bad* basis the cells are long thin slivers and rounding lands far from the true nearest point — the search blows up with the dimension n (real schemes use n in the hundreds or thousands).

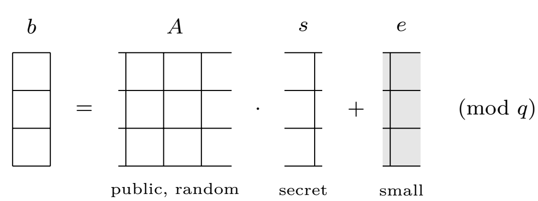
Crucially, these problems have *no hidden periodicity* for Shor's algorithm to grab (§24); the best known quantum algorithms barely beat the classical ones. That absence of structure is exactly why lattices are the leading post-quantum foundation. (A sibling problem, **SIS** — short integer solution: find a short nonzero combination of public vectors summing to $0 \bmod q$ — underpins lattice *signatures*.)

27.3 Learning With Errors (LWE)

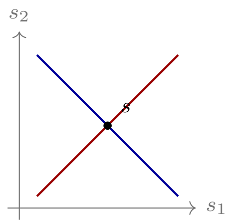
The problem actually used in practice hides a secret behind *noise*. Fix a modulus q and a secret vector $s \in \mathbb{Z}_q^n$. Publish a random matrix A and the vector

$$b = A\,s + e \pmod{q},$$

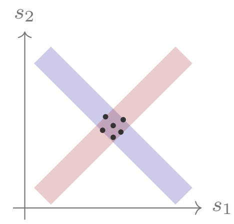
where e is a vector of *small* random errors. Given (A, b), recover s.



The single insight: *without* the noise, $b = As$ is an ordinary linear system, solved instantly by Gaussian elimination. The small e is what makes it hard. Each sample $\langle a, s\rangle = b$ is a line in the space of unknowns s; two clean lines cross at the single true s, but once each b may be off by $\pm e$, every line thickens into a *band*, and the bands overlap in a patch holding many candidate s:



no noise: the lines meet at one s



with noise: each line is a band — many s fit

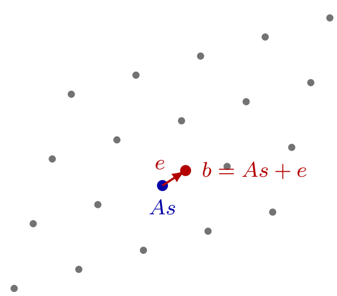
So you can no longer tell which s produced b — many do, almost.

Toy ($q = 17$, secret column $s = (3,1)^\top$; each a below is a row of A). Two published samples, each $b = \langle a, s\rangle + e$:

$$a = (5,2),\ e = 1 :\ b = 15 + 2 + 1 = 18 \equiv 1; \qquad a = (1,4),\ e = -1 :\ b = 3 + 4 - 1 = 6.$$

Noise-free, the pairs $(5,2) \to 0$ and $(1,4) \to 7$ would give two linear equations that pin down $s = (3,1)$ at once. The ± 1 wobble is everything: at n in the hundreds, no method recovers s.

Why this is a lattice problem. The rows of A together with q define a lattice; the exact product As is one of its points, and b sits just off it, displaced by the tiny e. Finding s is finding that nearby lattice point — a CVP instance (§27) with a guaranteed-small offset, called *bounded distance decoding*.



Recovering s means snapping the public point b back to the lattice point As nearest it; the small e is the guarantee that the right point is close by.

27.4 Encryption from LWE (Regev)

LWE gives public-key encryption directly. Sizes: A is $m \times n$, the secret $s \in \mathbb{Z}_q^n$, the noise $e \in \mathbb{Z}_q^m$.

Keys. Private s; public $(A,\ b = As + e)$.

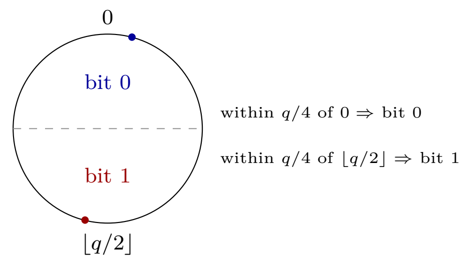
Encrypt a bit $\mu \in \{0,1\}$. Pick a random 0/1 vector $r \in \{0,1\}^m$ and send

$$u = A^\top r \pmod q, \qquad v = b^\top r + \mu \left\lfloor \tfrac{q}{2} \right\rfloor \pmod q.$$

Decrypt. Compute $v - s^\top u$ and expand, using $b = As + e$:

$$v - s^\top u = b^\top r - s^\top A^\top r + \mu \left\lfloor \tfrac{q}{2} \right\rfloor = \underbrace{(b - As)^\top r}_{= e^\top r,\ \text{small}} + \mu \left\lfloor \tfrac{q}{2} \right\rfloor \approx \mu \left\lfloor \tfrac{q}{2} \right\rfloor.$$

The cross term $s^\top A^\top r = (As)^\top r$ cancels the bulk of $b^\top r$, leaving only the small $e^\top r$ plus the message term. Round to decide the bit:



As long as the noise $e^\top r$ stays under $q/4$, the value lands in the correct half and μ comes back exactly. That tension — noise big enough to hide s, small enough to decrypt — is the dial every lattice scheme tunes.

27.5 Making it practical: Ring and Module LWE

Plain LWE keys are large: A alone is an $m \times n$ matrix of numbers mod q. Real schemes shrink it by giving A *structure* — its entries become polynomials in a ring $\mathbb{Z}_q[x]/(x^n + 1)$, so one short polynomial stands in for a whole block of the matrix. This is *Ring-LWE*; the middle ground used by NIST's winners is *Module-LWE*.

Scheme	Lattice problem	Role	Where
Regev PKE (textbook)	LWE	teaching example	§13
ML-KEM / Kyber (FIPS 203)	Module-LWE	key agreement (KEM)	§11
ML-DSA / Dilithium (FIPS 204)	Module-LWE / SIS	signatures	§11
FrodoKEM (conservative)	plain LWE	key agreement	—

Kyber, for instance, works with degree-256 polynomials and a tiny modulus $q = 3329$, which is what lets its keys and ciphertexts fit in roughly a kilobyte — comparable to the classical schemes it replaces, while resisting Shor.

27.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\mathcal{L}$	a lattice: all integer combinations of the basis vectors
b_i / c_i	basis vectors (a good / bad basis of the same lattice)
SVP, CVP	shortest-vector and closest-vector problems
q	the modulus (arithmetic in $\mathbb{Z}_q$, §21)
$A,\ s,\ e,\ b$	public matrix, secret vector, small noise, and $b = As + e$
$\mu,\ r$	the plaintext bit and the random 0/1 encryption vector
$\lfloor q/2 \rfloor$	the "half" value that encodes the bit $\mu = 1$
Ring / Module-LWE	LWE over a polynomial ring (Kyber, Dilithium)

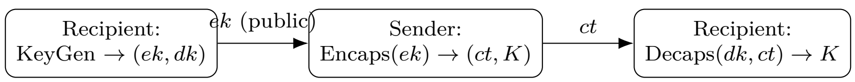
28 ML-KEM (Kyber)

ML-KEM is NIST's post-quantum key-agreement standard (FIPS 203, 2024), the scheme formerly called *CRYSTALS-Kyber*. The name is **M**odule-**L**attice **KEM** — *not* machine learning. It is the drop-in replacement for the (EC)DH key exchange of §8 that Shor breaks (§24), and it is built on the Module-LWE problem of §27.

It comes in two layers, and the section follows them: an LWE *encryption* scheme inside (§27's Regev idea, over modules), wrapped by a transform that upgrades it into a robust *KEM*.

28.1 The interface

A KEM does one job: let a sender hand a recipient a fresh shared secret K, using only the recipient's public key. Three routines:



Both sides end holding the same K, which feeds a KDF just as Diffie–Hellman's shared secret did (§8). Here *ek* is the public *encapsulation key*, *dk* the secret *decapsulation key*, and *ct* the *ciphertext* (the encapsulation). The goal is security even against an attacker who can feed chosen ciphertexts to Decaps — which is what the second layer buys.

28.2 The ring it lives in

ML-KEM computes not with plain numbers but with *polynomials*. A *ring element* is a polynomial of degree below 256 whose coefficients live in $\mathbb{Z}_q$ with $q = 3329$ (§21); two are added coefficient-wise and multiplied modulo $x^{256} + 1$, so the degree never grows. Write

$$R_q = \mathbb{Z}_q[x] \,/\, (x^{256} + 1).$$

The *module* in Module-LWE means working with short vectors and a matrix whose entries are such ring elements. The *rank* $k \in \{2, 3, 4\}$ sets the strength (more below). Polynomial multiplication is made fast by the *number-theoretic transform* (NTT) — an FFT over $\mathbb{Z}_q$ — but that is a speed trick, not part of the security.

28.3 The encryption inside (K-PKE)

This is §27's Regev scheme, now over R_q. The matrix $A \in R_q^{k \times k}$ and the vectors s, e are as in LWE, only their entries are ring elements and s, e are *short*.

Keys. Sample short s, e; set $t = A\,s + e$. The public key is (ρ, t) and the secret key is s, where ρ is the 32-byte *seed* that regenerates A — A itself is never stored, only re-expanded from ρ by SHAKE, which is what keeps the key near a kilobyte instead of a megabyte.

Encrypt a 256-bit message m (one bit per polynomial coefficient): draw short r, e_1 and short e_2, and send

$$u = A^{\top} r + e_1, \qquad v = t^{\top} r + e_2 + \left\lfloor \tfrac{q}{2} \right\rceil m.$$

Decrypt: form $w = v - s^{\top} u$ and round each coefficient (near $0 \to$ bit 0, near $\lfloor q/2 \rceil \to$ bit 1). The cancellation is exactly §27's, with a few more small terms:

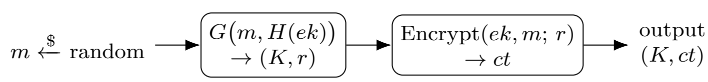
$$v - s^\top u = (As+e)^\top r + e_2 + \lfloor \tfrac{q}{2} \rceil m - s^\top (A^\top r + e_1) = \lfloor \tfrac{q}{2} \rceil m + \underbrace{\left(e^\top r - s^\top e_1 + e_2\right)}_{\text{small}},$$

because $(As)^\top r = s^\top A^\top r$ cancels. The leftover noise stays under $q/4$, so rounding returns m unharmed.

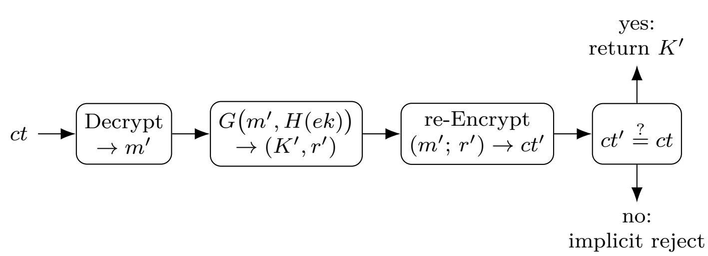
28.4 From PKE to KEM: the Fujisaki–Okamoto transform

The encryption above is only safe against a passive eavesdropper. A KEM must resist an attacker who submits *tampered* ciphertexts and watches what comes back. The fix (the *FO transform*) is to make the randomness deterministic and *re-encrypt on the way in* to check the ciphertext was honestly formed.

Encaps(ek)*:* pick a random 32-byte m, hash it to get both the shared key and the encryption randomness, then encrypt m with that randomness:



Decaps(dk, ct)*:* decrypt to m', re-derive (K', r') the same way, *re-encrypt*, and accept only if the ciphertext reproduces exactly:



Because r comes from hashing m, an honest ciphertext is the *only* one that re-encrypts to itself; any tampering fails the check. On failure Decaps does not error — it returns a pseudorandom *implicit-reject* key (a hash of ct and a secret), so the attacker learns nothing from the difference. This is what lifts passive (chosen-plaintext) security to the active (chosen-ciphertext) security a KEM needs.

All the hashing here is the SHA-3 / SHAKE sponge from the hashing collection: SHAKE expands ρ into A, G is SHA3-512 (producing K and r), H is SHA3-256.

28.5 Parameters and practicalities

One ring ($q = 3329$, degree 256) for all sizes; only the module rank k changes:

Scheme	rank k	NIST level	public key ek	ciphertext ct
ML-KEM-512	2	1	800 B	768 B
ML-KEM-768	3	3	1184 B	1088 B
ML-KEM-1024	4	5	1568 B	1568 B

Two practical notes. There is a tiny chance the leftover noise exceeds $q/4$ and decryption fails; parameters are chosen so this probability is negligible (below 2^{-138}), far rarer than a hardware fault. And in deployment ML-KEM is run *hybrid* — next to a classical X25519, both secrets folded into the KDF (§11) — so a future break of either scheme alone is survivable.

28.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$ek,\ dk$	encapsulation (public) and decapsulation (secret) keys
$ct,\ K$	the ciphertext (encapsulation) and the shared secret it carries
R_q	the ring $\mathbb{Z}_q[x]/(x^{256}+1)$, $q = 3329$
k	module rank: $2, 3, 4$ for ML-KEM-512/768/1024
$A,\ s,\ e,\ t$	public matrix (from seed ρ), short secret, short noise, $t = As + e$
ρ	32-byte seed that regenerates A via SHAKE
$m,\ r$	the random seed that is encapsulated, and the derived encryption randomness
$G,\ H$	SHA3-512 (yields K, r) and SHA3-256 (hashes ek)
FO transform	the re-encrypt-and-check wrapper giving chosen-ciphertext security

29 ML-DSA (Dilithium)

ML-DSA is NIST's post-quantum *signature* standard (FIPS 204, 2024), formerly *CRYSTALS-Dilithium*. It is the partner of ML-KEM (§28): same module-lattice world, but it signs rather than agrees keys, replacing the RSA/ECDSA/EdDSA signatures (§9, §18) that Shor breaks (§24). It rests on two lattice problems from §27: Module-LWE hides the secret key, and Module-SIS makes forgery hard.

The cleanest way in is that ML-DSA is a lattice rebuild of the Schnorr/EdDSA scheme of §18 — commit, challenge, respond — so we lead with that parallel.

29.1 The interface

The same three routines as any signature (§9):

Signer: KeyGen $\to (pk, sk)$ $\xrightarrow{pk}$ Signer: Sign$(sk, M) \to \sigma$ $\xrightarrow{(M,\sigma)}$ Anyone: Verify(pk, M, σ) $\longrightarrow$ accept / reject

29.2 Keys: a Module-LWE instance

The key pair is exactly an LWE sample (§27). Take a public matrix A of ring elements (re-expanded from a seed, as in §28) and two *short* secret vectors s_1, s_2; the public key is

$$t = A\,s_1 + s_2.$$

Recovering s_1, s_2 from (A, t) is the Module-LWE problem — believed hard for quantum computers. So $pk = (A_{\text{seed}}, t)$ is public, $sk = (s_1, s_2)$ is secret.

29.3 Sign and verify: Fiat–Shamir with aborts

The scheme follows the Schnorr/EdDSA shape of §18 step for step, with vectors and matrices in place of curve points:

Step	Schnorr / EdDSA (§18)	ML-DSA
commitment	$R = kG$	$w = A\,y$ (y short, random)
challenge	$c = H(R, M)$	$c = H\big(\text{HighBits}(w),\, M\big)$
response	$s = k + c\,d$	$z = y + c\,s_1$
verify	$R \stackrel{?}{=} sG - cQ$	$\text{HighBits}(Az - c\,t) \stackrel{?}{=} \text{HighBits}(w)$
secret hidden by	discrete log	Module-LWE
extra step	—	*abort* if z too large (below)

Here y is a fresh short *masking* vector (Schnorr's one-time k), and the challenge c is a small polynomial with a handful of ± 1 coefficients.

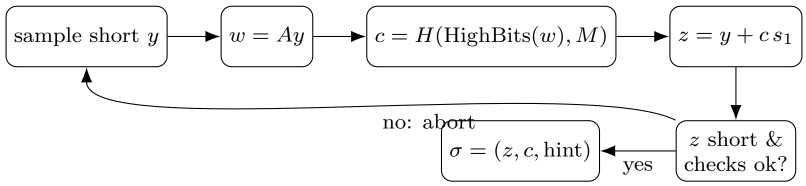
Why verification works. Substitute $t = As_1 + s_2$ and $z = y + c\,s_1$:

$$Az - c\,t = A(y + c\,s_1) - c(As_1 + s_2) = Ay - c\,s_2 = w - c\,s_2 \;\approx\; w.$$

The As_1 terms cancel, exactly as cdG cancels in Schnorr's $sG - cQ = kG$. What remains differs from w only by the *small* $c\,s_2$, so the coarse **high bits** are unchanged: $\text{HighBits}(Az - c\,t) = \text{HighBits}(w)$, and recomputing $c = H(\text{HighBits}(\cdot), M)$ reproduces the challenge. (A tiny *hint* in σ fixes the rare coefficient that sits on a rounding boundary.)

29.4 Why the aborts

There is one twist Schnorr does not have. The response $z = y + c\,s_1$ contains the secret s_1; if it were released as-is, its size would leak information about s_1 over many signatures. So ML-DSA uses *rejection sampling*: it outputs z only when it lands in a fixed safe range, and otherwise throws away y and restarts. That keeps the published z looking independent of the secret — this is the "with aborts".



A signature takes a few attempts on average (the loop runs a handful of times), the only real cost of the rejection trick.

29.5 Parameters and cost

Ring degree 256 again, but a larger prime $q = 8\,380\,417$; the strength comes from the matrix dimensions (k, ℓ), which the suffix names:

Scheme	(k, ℓ)	NIST level	public key / signature
ML-DSA-44	$(4, 4)$	2	1312 B / 2420 B
ML-DSA-65	$(6, 5)$	3	1952 B / 3309 B
ML-DSA-87	$(8, 7)$	5	2592 B / 4627 B

The headline cost is size: an ML-DSA signature is a few kilobytes, against 64 bytes for Ed25519 (§18). That bloat — shared by every lattice signature — is the price of quantum resistance. As with ML-KEM, all the hashing (expanding A, computing c) is the SHA-3 / SHAKE sponge of the hashing collection, and deployments often pair ML-DSA with a classical signature during the transition (§11).

29.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
pk, sk	public and secret keys
A	public matrix of ring elements (from a seed, as in §28)
s_1, s_2	short secret vectors; $t = As_1 + s_2$ is the public key
y	short one-time masking vector (Schnorr's k, §18)
w	commitment Ay; $\mathrm{HighBits}(w)$ is its coarse part
c	challenge: a small ± 1 polynomial, $c = H(\mathrm{HighBits}(w), M)$
z	response $z = y + c\,s_1$
σ	the signature (z, c, hint)
Module-SIS	the short-solution lattice problem that makes forgery hard

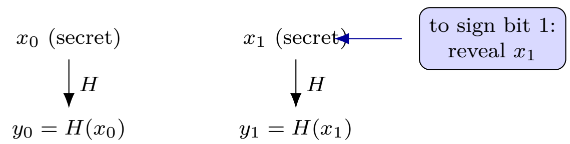
30 SLH-DSA (SPHINCS$^+$)

SLH-DSA is the third NIST post-quantum standard (FIPS 205, 2024), from SPHINCS$^+$. It signs, like ML-DSA (§29), but on a completely different footing: its security rests on *nothing but a hash function* — no number theory, no lattices. That is the most conservative assumption in all of cryptography, which is why it is the *backup*: if some future attack broke lattices, a good hash would very likely still stand (the worst a quantum computer does to a hash is Grover's square-root, §25, fixed by a longer output).

It is built bottom-up from three ideas, and this section climbs them in order: a one-time signature, a Merkle tree to get many signatures from one key, and a stateless hypertree to make it safe to deploy.

30.1 The brick: a one-time signature

The *Lamport* one-time signature shows the whole idea. To sign a single bit, keep two random secrets x_0, x_1; publish their hashes $y_0 = H(x_0)$, $y_1 = H(x_1)$. To sign the bit b, simply reveal x_b; the verifier checks $H(x_b) = y_b$.

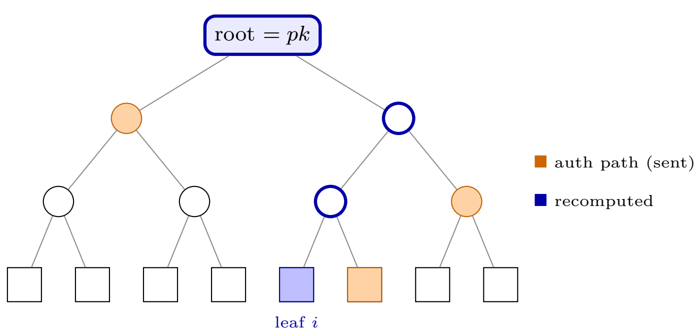


The public key is (y_0, y_1); for an n-bit message you repeat this n times. It works once and *only* once — sign two different messages and you reveal enough secrets to forge a third. (SPHINCS$^+$ actually uses a more compact one-time scheme, *WOTS*$^+$, where the public key is the end of a hash chain $a \to H(a) \to H^2(a) \to \cdots$ and you sign a digit by revealing a point partway along; same one-time nature, smaller keys.)

30.2 Many signatures from one key: the Merkle tree

One public key is good for one signature — useless. A *Merkle tree* fixes this. Generate 2^h one-time key pairs, hash each one-time public key into a *leaf*, then hash leaves in pairs up a binary tree until a single *root* remains. That root — one short hash — is the public key for all 2^h one-time keys at once.

To sign message number i, use the i-th one-time key and attach the *authentication path*: the sibling hash at each level, just enough for the verifier to recompute the root and check it matches the public key.



From leaf i and the three orange siblings the verifier rebuilds the blue path up to the root. This gives 2^h signatures from one public key — but it is **stateful**: you must remember which leaves you have used, because reusing one reuses a one-time key and breaks everything.

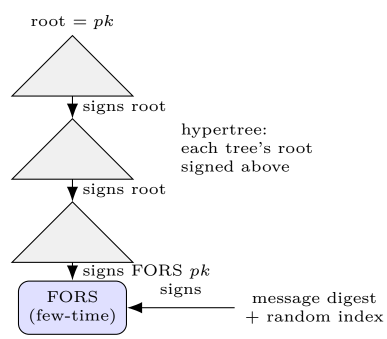
30.3 Going stateless: random index, hypertree, FORS

State is a deployment trap — a restored backup or cloned VM can silently reuse an index. SPHINCS$^+$ removes the state with three moves:

- **Random index.** Make the tree enormous (height 60 and up) and pick the leaf *pseudo-randomly* from the message and a secret. With 2^{60} leaves, an accidental repeat is negligible — no counter to keep.
- **Hypertree.** One tree of height 60 would be impossibly slow to build. Instead use a *tree of trees*: several layers of short Merkle trees, where each tree's root is signed by a one-time key in the tree above. Only the single chain of trees down to the chosen leaf is ever computed.
- **FORS.** The bottom one-time key does not sign the message; it signs the public key of a *few*-time signature called FORS, and FORS signs the actual message digest. FORS tolerates the rare random-index clash that would sink a strict one-time key.

30.4 The whole signature

Stacking those: the message is hashed (with randomness) into a digest and a leaf index; FORS signs the digest; the bottom hypertree tree signs the FORS key; and each tree's root is signed by the layer above, up to the top root — which is the public key.



The signature is the FORS signature plus, for every hypertree layer, a one-time signature and its authentication path. Verifying just re-hashes upward and checks the final root equals the public key — only hashing, start to finish.

30.5 Cost, and when to reach for it

The bill for assuming so little is size and speed: SLH-DSA signatures run from about 8 KB to over 40 KB, and signing is markedly slower than ML-DSA. Two "s" (small) and "f" (fast) variants trade signature size against signing time at each level:

Family	NIST level	public key	signature
SLH-DSA-128 (s / f)	1	32 B	∼7.9 / 17 KB
SLH-DSA-192 (s / f)	3	48 B	∼16 / 35 KB
SLH-DSA-256 (s / f)	5	64 B	∼29 / 50 KB

Note how tiny the public key is (just the root hash) against the large signature. The verdict: ML-DSA (§29) is the everyday default; SLH-DSA is the conservative hedge — used where a signature must stay verifiable for decades and you want it to depend on the single most-studied assumption available, a hash function. As ever, all of H is the SHA-2 or SHA-3 / SHAKE of the hashing collection.

30.6 Symbols

No new persistent symbols. Local to this section:

Local symbol / term	Meaning (this section)
H	a hash function — the sole security assumption
OTS	one-time signature (Lamport, or WOTS^+ in SPHINCS^+)
$x_b,\ y_b$	a Lamport secret and its public hash $y_b = H(x_b)$
Merkle tree	binary hash tree; its root is the public key
leaf / root	a hashed one-time public key / the tree's top hash
auth path	the sibling hashes that let a verifier rebuild the root
hypertree	tree of trees: each tree's root signed by the layer above
FORS	few-time signature that signs the message digest

31 Zero-knowledge proofs

A zero-knowledge proof lets a *prover* convince a *verifier* that a statement is true while revealing *nothing else* — not even why it is true. Prove you know a password without sending it; prove you are over 18 without showing your birthdate; prove a computation was done correctly without redoing it.

We have, in fact, already built one. The Schnorr identification behind EdDSA (§18) is a zero-knowledge proof of knowledge of a secret key, and Fiat–Shamir (§29) is what turns such a proof into a signature. This section names the idea, makes that connection explicit, and then reaches the modern succinct proofs.

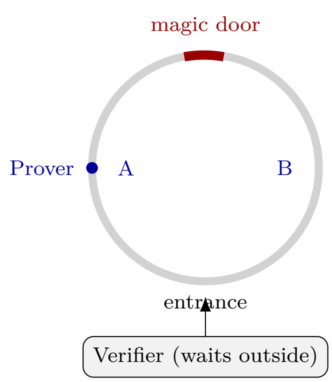
31.1 The three properties

A protocol is a zero-knowledge proof when it has all three of:

- **Completeness** — if the statement is true and both parties are honest, the verifier accepts.
- **Soundness** — if the statement is false, no cheating prover can make the verifier accept, except with tiny probability.
- **Zero-knowledge** — the verifier learns nothing beyond "true". Formally: a *simulator* with no secret can produce a transcript indistinguishable from a real one, so the transcript cannot have taught the verifier anything.

31.2 The intuition: the cave

The classic picture is a ring-shaped cave with a magic door at the far end that opens only to a secret word. The prover claims to know the word.

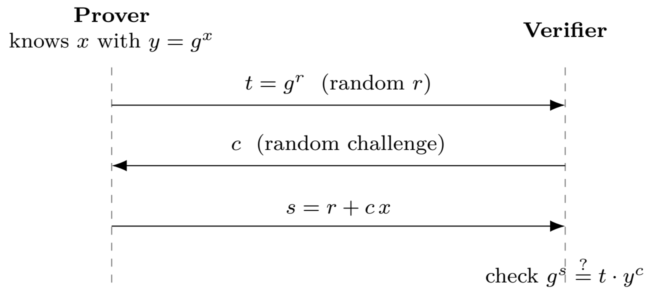


Each round: the prover walks in and down one side (A or B) out of the verifier's sight; the verifier then shouts which side to come out. Knowing the word, the prover can always cross through the door and exit on the demanded side. A faker who picked the wrong side is stuck — they only escape if they guessed the verifier's call, a $1/2$ chance.

So one round catches a cheat half the time; n rounds drop a faker's success to 2^{-n} (soundness). An honest prover always passes (completeness). And the verifier, who only ever sees the prover emerge from the side they themselves named, learns nothing about the word (zero-knowledge) — they could have scripted the whole exchange alone.

31.3 A real one: Schnorr's protocol

The cryptographic version proves *knowledge of a discrete logarithm* (§22). The public statement is $y = g^x$; the prover knows the secret x (its *witness*) and reveals nothing about it. It is a three-move exchange — commit, challenge, respond (a Σ-*protocol*):



Completeness. If the prover is honest the check holds by the exponent algebra:

$$g^s = g^{\,r+cx} = g^r\,(g^x)^c = t \cdot y^c.$$

Soundness. Suppose a prover could answer *two* different challenges $c \neq c'$ for the *same* commitment t, with responses s, s'. Dividing $g^s = ty^c$ by $g^{s'} = ty^{c'}$ gives $g^{\,s-s'} = y^{\,c-c'}$, hence

$$x = \frac{s - s'}{c - c'} \pmod{q}.$$

So anyone who can reliably answer is forced to *know* x — a cheater without it can satisfy at most one challenge, and guesses the rest with negligible odds.

Zero-knowledge. The transcript (t, c, s) leaks nothing, because it can be forged without x: pick s and c at random and set $t = g^s y^{-c}$. The check passes by construction and the fake is distributed exactly like a real run — so seeing a real transcript taught the verifier nothing it could not have written itself.

31.4 Non-interactive proofs: Fiat–Shamir

The protocol above needs the verifier live, to send a fresh random c. *Fiat–Shamir* removes them: replace the challenge with a hash of the commitment and the statement,

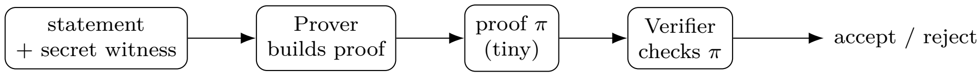
$$c = H(t,\ y,\ \text{message}),$$

so the prover generates their own unguessable challenge and the proof collapses to a single message (t, s) anyone can check. A hash the prover cannot control stands in for the verifier's coin.

This is precisely a digital signature. A Schnorr/EdDSA signature (§18) is this non-interactive proof with the signed message folded into c; ML-DSA (§29) is the same move over lattices. *A signature is a non-interactive zero-knowledge proof that you know the secret key, bound to a message.*

31.5 Modern zero-knowledge: SNARKs and STARKs

Schnorr proves one fixed fact (knowledge of a discrete log). The breakthrough of the last decade is proving *arbitrary* statements — "I ran this program on a secret input and got this output" — with a proof far smaller than the computation itself. The statement is written as an arithmetic circuit; the secret input is the *witness*; the prover shows it knows a witness satisfying the circuit.



The verifier never sees the witness, and checking π is far cheaper than rerunning the computation. Two families dominate:

	zk-SNARK	zk-STARK
Name	*succinct* argument	*scalable, transparent* argument
Proof size	tiny (a few hundred bytes)	larger (tens of KB)
Trusted setup	yes ("toxic waste" to discard)	none (transparent)
Built on	elliptic-curve pairings	hashes only
Post-quantum	no (curves, §24)	yes (hash-based)

These power private transactions (prove a payment is valid while hiding amounts and parties) and *rollups* (prove a whole batch of transactions was processed correctly with one small proof, so verifiers need not replay them) — and selective disclosure, like proving you are over 18 from a signed credential without revealing the date.

31.6 Symbols

No new persistent symbols. Local to this section:

Local symbol / term	Meaning (this section)
completeness / soundness / ZK	the three defining properties
$y = g^x$	the public statement; x is the secret *witness*
$t = g^r$	the prover's commitment (r a fresh random nonce)
c	the verifier's challenge
$s = r + c\,x$	the prover's response
Σ-protocol	a three-move commit–challenge–respond proof
Fiat–Shamir	set $c = H(t, \dots)$ to make the proof non-interactive
zk-SNARK / zk-STARK	succinct / transparent proofs for arbitrary computations